
Software Requirements

Software/Tool	Version	Purpose	Installation Notes
Java JDK	17.0 or above (preferred)	Required for building and running Java-based applications.	Download from Oracle or OpenJDK distributions.
IntelliJ IDEA	2022.2 or above	Integrated Development Environment (IDE) for Java development and project management.	Download from JetBrains .
Eclipse For Developers	2022-09 or above	Integrated Development Environment (IDE) for Java development and project management.	Download from Eclipse Packages .
Apache Tomcat Server	10 or above	Local Server For Basic WebSite Deployment.	Download from Apache Tomcat .

Git	Latest stable version	Version control system for managing source code and collaborating via GitHub.	Download from Git .
Maven	Latest stable version	Build automation tool for Java projects.	Download from Apache Maven .
Gradle	Latest stable version	Alternative build automation tool for managing dependencies and tasks.	Download from Gradle .
Jenkins	Latest stable version	CI/CD tool for automating builds, tests, and deployments.	Download from Jenkins .
Oracle VM VirtualBox	Latest stable version	Virtualization tool for creating and managing virtual machines.	Download from VirtualBox .
Vagrant	Latest stable version	Tool for managing and provisioning virtual machine environments.	Download from Vagrant .
Ansible	Latest stable version	Configuration management and automation tool.	Install via Ansible documentation .
Microsoft Azure	Free Tier Account	Needed for DevOps CI-CD Pipelines & Deployments.	Install via Microsoft Azure SignUp .

Pre-requisites and Software Requirements

Pre-requisites

- Java Programming Knowledge:**
 - Basic understanding of Java syntax, OOP concepts, and familiarity with building Java applications.
- Operating System:**
 - Windows, macOS, or Linux (64-bit recommended).
- Internet Connectivity:**
 - Reliable internet connection for downloading software, accessing GitHub, and Azure resources.
- GitHub Account:**
 - All participants must have a fully functional **GitHub account** for version control and source code management.



Experiment 1: Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup

1. Understanding DevOps

What is DevOps?

DevOps is a set of cultural philosophies, practices, and tools that combine software development (Dev) and IT operations (Ops). Its goal is to shorten the systems development life cycle while delivering features, fixes, and updates frequently in close alignment with business objectives. DevOps promotes:

- **Collaboration:** Breaking down silos between development and operations teams.
- **Automation:** Automating repetitive tasks (builds, tests, deployments) to improve efficiency.
- **Continuous Integration/Delivery (CI/CD):** Regularly integrating code changes and deploying them to production with minimal manual intervention.
- **Monitoring and Feedback:** Constantly monitoring the performance and behavior of applications in production to rapidly address issues.

Why is DevOps Used?

- **Speed and Agility:** Faster development cycles and quicker time-to-market.
- **Quality:** Automated testing and integration help catch issues early.
- **Reliability:** Frequent, smaller updates reduce the risk of large-scale failures.
- **Efficiency:** Automation reduces manual errors and repetitive tasks.
- **Scalability:** Processes and infrastructure can grow with the business.

Examples of DevOps in Action:

- **Continuous Integration (CI):** Tools like Jenkins or Azure Pipelines automatically build and test code on every commit.
- **Continuous Deployment (CD):** Systems automatically deploy tested code to production environments.
- **Monitoring:** Tools such as Prometheus, Grafana, or New Relic continuously monitor application performance.

2. Introduction to Maven and Gradle

What is Maven?

Maven is a build automation and project management tool primarily used for Java projects. It uses a central configuration file known as the **POM (Project Object Model)**, written in XML, which defines the project structure, its dependencies, build order, and plugins.

Key Features and Roles of Maven:

- **Convention over Configuration:** Maven enforces a standard directory structure (e.g., `src/main/java`, `src/test/java`), which simplifies project setup.
- **Dependency Management:** Automatically downloads and manages external libraries and dependencies from repositories like Maven Central.
- **Build Lifecycle:** Defines a fixed lifecycle (e.g., compile, test, package, install, deploy) which standardizes the build process.
- **Plugin Ecosystem:** Provides a rich set of plugins to extend functionality (e.g., unit testing, code coverage, reporting).

What is Gradle?

Gradle is a modern build automation tool that is known for its flexibility and performance. It uses a Groovy or Kotlin DSL (Domain Specific Language) to define build logic, which allows for more dynamic and customizable configurations compared to Maven's XML-based approach.

Key Features and Roles of Gradle:

- **Flexible Build Scripts:** Instead of a rigid XML file, Gradle build scripts written in Groovy or Kotlin allow you to include logic, conditionals, and loops.
- **Incremental Builds:** Gradle tracks changes in source files and only rebuilds what is necessary, which can significantly speed up the build process.
- **Multi-project Builds:** Easily manages complex projects with multiple modules or subprojects.
- **Extensibility:** A robust plugin system that enables you to integrate various languages, frameworks, and tools.
- **Parallel Execution:** Can run tasks in parallel, optimizing build times for large projects.

3. Why Use Maven and Gradle?

Both Maven and Gradle are used to automate the build process and manage project dependencies. Here's why they are so popular in the DevOps ecosystem:

- **Automated Build and Testing:** They allow developers to compile code, run tests, and package applications without manual intervention.
- **Consistent Build Environment:** By enforcing standardized project structures and dependency management, they help avoid the "it works on my machine" problem.
- **Ease of Integration:** Both tools integrate well with Continuous Integration (CI) servers like Jenkins, enabling automated pipelines.
- **Dependency Resolution:** They simplify the process of managing external libraries and ensure that all developers are using the same versions.

5. Detailed Differences Between Maven and Gradle

Below is a detailed comparison highlighting their main differences:

Aspect	Maven	Gradle
Build Script Language	Uses an XML-based configuration file (pom.xml).	Uses a DSL based on Groovy or Kotlin (build.gradle or build.gradle.kts).
Configuration Style	Declarative and rigid – follows strict conventions (convention over configuration).	Flexible and dynamic – allows you to write custom logic and conditions within the build script.
Build Lifecycle	Provides a fixed lifecycle (e.g., validate, compile, test, package, install, deploy).	Uses a task-based approach where tasks can be defined, customized, and linked in a flexible manner.
Dependency Management	Manages dependencies through the POM file; downloads them from central repositories (e.g., Maven Central).	Similar dependency management; supports dynamic version resolution and customizable dependency configurations.

Performance	Generally slower for large projects because of the fixed build lifecycle and less emphasis on incremental builds.	Often faster, thanks to incremental builds, caching, and parallel execution of tasks.
Extensibility & Plugins	Rich ecosystem of plugins but customization can be more challenging due to XML's verbosity and limitations in scripting.	Highly extensible through its scripting capabilities; writing custom tasks or plugins in Groovy/Kotlin is more straightforward.
Multi-Project Builds	Handles multi-module projects well but requires a strict directory layout and a parent POM for aggregating modules.	Excels in multi-project builds with simple configuration, allowing each subproject to be configured in a flexible manner.
Learning Curve	Easier for beginners due to its structured, convention-based approach but can become complex with large configurations.	More flexible but may have a steeper learning curve initially if you need to leverage its dynamic features and custom logic.
Community & Maturity	Has been around longer, so many legacy projects and extensive documentation exist.	Relatively newer, it has gained popularity due to its modern features, especially in Android and multi-language projects.
Integration with CI/CD	Well-supported by most CI/CD tools (like Jenkins, Azure Pipelines) with stable, predictable behavior.	Also integrates seamlessly with CI/CD tools and is often chosen for its faster build times and flexibility in pipeline scripting.

Java SE 17 Archive Downloads (17.0.12 and earlier)

Go to the [Oracle Java Archive](#) page.

This page includes archive downloads for JDK 17 updates 17.0.12 and earlier. The archive downloads for JDK 17 updates 17.0.13 and greater are offered under different license terms, and can be found [here](#).

The JDK is a development environment for building applications using the Java programming language.

The JDK includes tools useful for developing and testing programs written in the Java programming language and running on the Java™ platform.

WARNING: Older versions of the JDK are provided to help developers debug issues in older systems. **They are not updated with the latest security patches and are not recommended for use in production.**

For production use Oracle recommends downloading the latest JDK version.

Only developers and enterprise administrators should download these releases.

For current Java releases, please visit [Oracle Java SE Downloads](#).

Welcome

License

ABOUT MAVEN

What is Maven?

Installation

Downloads

Use

Configure

Release Notes

Security Reports

DOCUMENTATION

Maven Plugins

Maven Extensions

Maven Tools

Maven Daemon

Maven Upgrade Tool

Maven Wrapper

Index (category)

User Manual

Maven Repositories

Plugin Development

Contribute to Maven

Downloading Apache Maven and Maven Daemon

The following artifacts are available for download.

- Apache Maven
 - Current Maven 3.9.14
 - Preview Maven 4.0.0-rc-5
- Maven Daemon
 - Current Maven Daemon 1.0.5
 - Preview Maven Daemon 2.0.0-rc-3

Apache Maven

Apache Maven 3.9.14

Apache Maven 3.9.14 is the latest release. It is the recommended version for all users.

Maven 3.9+ requires JDK 8 or above to execute. It still allows you to build against 1.3 and other JDK versions by using toolchains.

Maven is distributed in several formats for your convenience. Simply pick a ready-made binary distribution archive and follow the installation instructions. Use a source archive if you intend to build Maven yourself.

	Link	Checksums	Signature
Binary tar.gz archive	apache-maven-3.9.14-bin.tar.gz	apache-maven-3.9.14-bin.tar.gz.sha512	apache-maven-3.9.14-bin.tar.gz.asc
Binary zip archive	apache-maven-3.9.14-bin.zip	apache-maven-3.9.14-bin.zip.sha512	apache-maven-3.9.14-bin.zip.asc
Source tar.gz archive	apache-maven-3.9.14-src.tar.gz	apache-maven-3.9.14-src.tar.gz.sha512	apache-maven-3.9.14-src.tar.gz.asc
Source zip archive	apache-maven-3.9.14-src.zip	apache-maven-3.9.14-src.zip.sha512	apache-maven-3.9.14-src.zip.asc

Build Tool

Learn

Support

News

Gradle Technologies

About



Watch our latest training videos on [YouTube](#).

Releases

Here you can find binaries and reference documentation for current and past versions of Gradle. You can find the next [release candidate](#) or a bleeding edge nightly build for the [release](#) and [master](#) branches on their respective pages.

You can [install Gradle](#) through various other tools, or download a ZIP using the links on this page.

Command-line completion scripts for bash and zsh can be downloaded from the [gradle-completion project page](#).

Getting Started Resources

There are many [Gradle Build Tool trainings](#) available to help you get started quickly. The new free courses are a part of the [DPE University](#) by Gradle Technologies.

The Gradle team offers free [training](#)

9.4.1

Installation and Environment Setup for Maven on Windows

1. Download Maven:

- Visit the Apache Maven official website: <https://maven.apache.org/>
- Download the latest version of Maven in zip format.

2. Extract the Files:

- Extract the contents of the downloaded zip file to a directory of your choice, e.g., `C:\Program Files\Maven`

3. Set Up Environment Variable:

- Right-click on the **Computer** icon on your desktop, select **Properties**.
- Click on **Advanced system settings**.
- In the System Properties window, click on the **Environment Variables** button.
- In the System Variables section, find and select the `Path` variable, then click on **Edit**.
- Add the Maven bin directory to the path:
`C:\Program Files\Maven\apache-maven-3.x.x\bin`
- Click **OK** to close all dialogs.

Installation and Environment Setup for Gradle on Windows

1. Download Gradle:

- Visit the Gradle Releases page: <https://gradle.org/>
- Download the latest version of Gradle in zip format.

2. Extract the Files:

- Extract the contents of the downloaded zip file to a directory of your choice, e.g., `C:\Program Files\Gradle`.

3. Set Up Environment Variable:

- Follow the same steps as for **Maven** to open the System Properties and Environment Variables dialog.
- Add the Gradle bin directory to the Path variable:

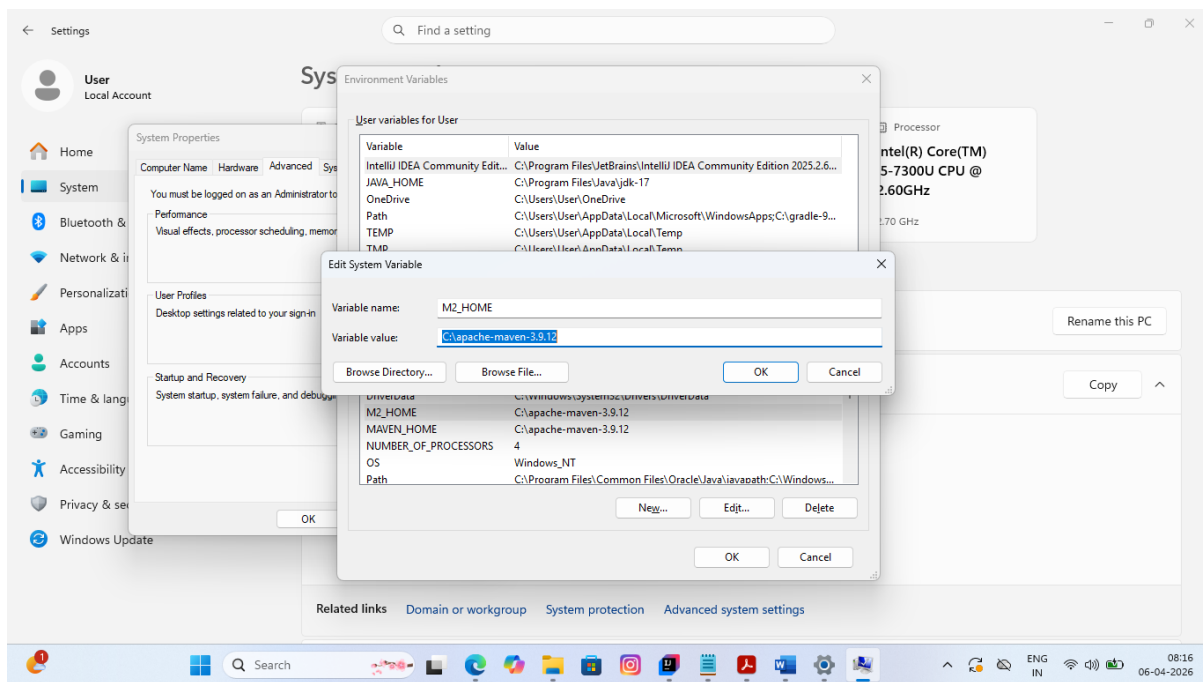
`C:\Program Files\Gradle\gradle-6.x\bin.`

- Ensure all dialogs are closed by clicking **OK**.

4. Verify Installation:

Open Command Prompt (cmd) and type:

```
gradle --version
```



```
Microsoft Windows [Version 10.0.26100.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>java --version
java 17.0.12 2024-07-16 LTS
Java(TM) SE Runtime Environment (build 17.0.12+8-LTS-286)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.12+8-LTS-286, mixed mode, sharing)

C:\Users\User>
```

```
C:\Users\User>mvn --version
Apache Maven 3.9.12 (848fbb4bf2d427b72bdb2471c22fced7ebd9a7a1)
Maven home: C:\apache-maven-3.9.12
Java version: 17.0.12, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-17
Default locale: en_IN, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

C:\Users\User>
```

```
Command Prompt
C:\Users\User>gradle --version

-----
Gradle 9.3.1
-----

Build time:    2026-01-29 14:15:01 UTC
Revision:     44f4e8d3122ee6e7cbf5a248d7e20b4ca666bda3

Kotlin:       2.2.21
Groovy:       4.0.29
Ant:          Apache Ant(TM) version 1.10.15 compiled on August 25 2024
Launcher JVM: 17.0.12 (Oracle Corporation 17.0.12+8-LTS-286)
Daemon JVM:   C:\Program Files\Java\jdk-17 (no Daemon JVM specified, using current Java home)
OS:           Windows 11 10.0 amd64

C:\Users\User>
```

Experiment 2: Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

1. Introduction to Maven

- **Maven** is a powerful build automation and project management tool primarily used for Java projects. It simplifies the build process by:
- Enforcing a **standard project structure** (convention over configuration).
- Managing **dependencies** automatically by downloading them from remote repositories (e.g., Maven Central).
- Defining a clear **build lifecycle** (compile, test, package, install, deploy).
- Allowing the integration of various **plugins** to extend functionality (e.g., testing, reporting).

Steps to create Maven project in IntelliJ IDEA

1. Install Maven

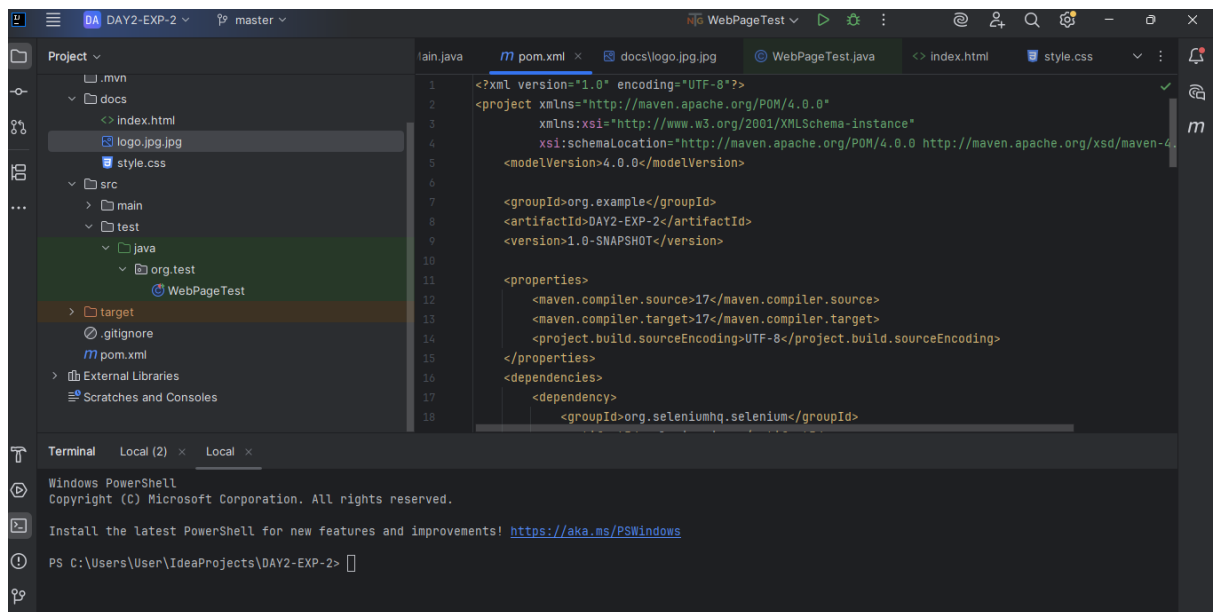
- Download Maven from the official website.
<https://maven.apache.org/download.cgi?>
- Set the Maven_Home environment variable and update the system PATH.

2. Create a New Maven Project :

- Open IntelliJ IDEA.
- Go to file >New >Project
- Select Maven from Build System
- Choose create from archetypes(optional) or proceed without.
- Set the project name and location, then click Finish.

3. Set Up the pom.xml File:

- The pom.xml file is where you define dependencies, plugins, and other configurations for your Maven project. Example of a basic pom.xml :



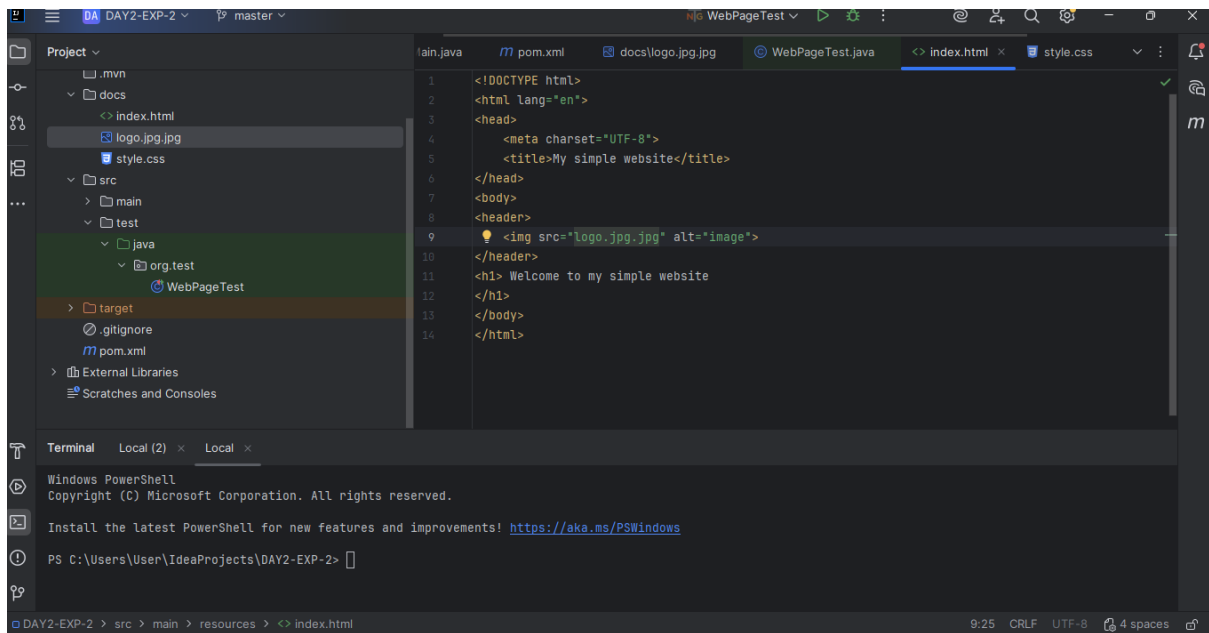
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                              http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
    <modelVersion>4.0.0</modelVersion>
```

```
    <groupId>com.example</groupId>
    <artifactId>my-app</artifactId>
    <version>1.0.0</version>
```

```
    <dependencies>
        <!--Add your dependencies here -->
    </dependencies>
```

```
</project>
```

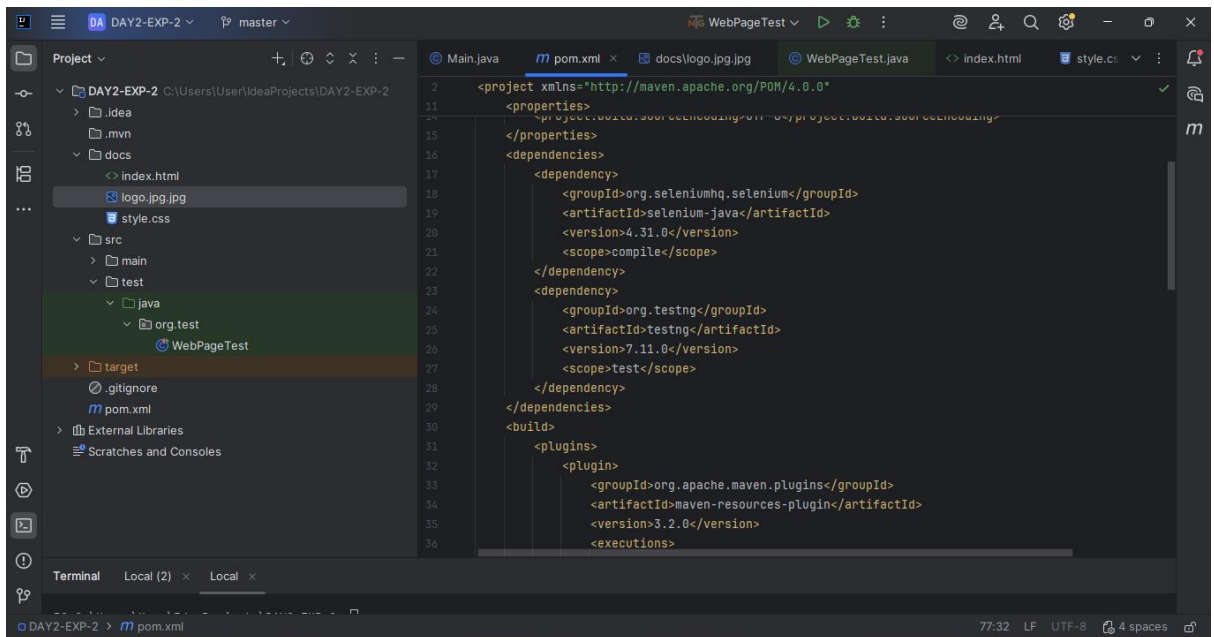



4. Add Dependencies for Selenium and TestNG:

- In the pom.xml file, add Selenium and TestNG dependencies under the <dependencies> section.
- Go to Browser-> search for maven repository ->search Selenium java

Add dependency similarly for testNG

- <dependencies>
 <dependency>
 <groupId>org.seleniumhq.selenium</groupId>
 <artifactId>selenium-java</artifactId>
 <version>4.31.0</version>
 </dependency>
- <dependency>
 <groupId>org.testng</groupId>
 <artifactId>testng</artifactId>
 <version>7.11.0</version>
 <scope>test</scope>
 </dependency>
</dependencies>



5. Create a simple website(Html , CSS and logo).

- In the src/main/resources folder, create an index.html file, style.css file, and place the logo.png image.

- Example of a simple index.html :

- `<!DOCTYPE html>`

`<html lang ="en">`

`<head>`

`<meta charset="UTF-8">`

`<meta name="viewport" content="width=device-width, initial-scale=1.0">`

`<title>My Simple Website</title>`

`<link rel="stylesheet" href="style.css">`

`</head>`

`<body>`

`<header>`

``

`</header>`

`<h1>Welcome to My Simple Website</h1>`

`</body>`

`</html>`

Example of a simple style.css :

```

    body {
font-family: Arial, sans-serif;

background-color: #f4f4f4;

text-align: center;

    }
header img {

width: 100px;

    }

```

6. Upload the website to Github.

- Initialize a Git repository in your project folder:

```

1 git init
2

```

- Add your files and commit them:

```

1 git add .
2 git commit -m "Initial commit"
3

```

- Create a GitHub repository and push the local project to GitHub:

```

1 git remote add origin <your-repository-url>

```

Deployment

- To deploy your Maven project to GitHub Pages using the /docs folder (** having all files inside root folder/dir not recommended), you can follow these simple steps.
- **Steps to Deploy to GitHub Pages Using /docs Folder**
 1. Modify Maven Configuration to Copy Static Files to your static files (index.html , style.css , /docs Folder: First, you need to ensure that Maven places logo.png) into the /docs folder instead of the target directory.
- Add Maven resources plugin in your pom.xml to copy the files directly into /docs



1. Maven Resources Plugins → to convert html,css and images to docs files.

Eg. 3.2.0 version

2. We have to write the code for execution to convert resources folder into docs

Folder.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-resources-plugin</artifactId>
      <version>3.2.0</version>
    <executions>
      <execution>
        <phase>prepare-package </phase>  <!-- Before packaging →
        <goals>
          <goal>copy-resources</goal>
        </goals>
        <configuration>
          <outputDirectory>${project.basedir}/docs</outputDirectory> <!-- deploy to   /docs
folder -->
          <resources>
            <resource>
              <directory>src/main/resources</directory>
              <includes>
                <include>**/*</include> <!--Copy all files in src/main/resources →
              </includes>
            </resource>
          </resources>
        </configuration>
      </execution>
    </executions>
  </plugin>
</plugins>
```

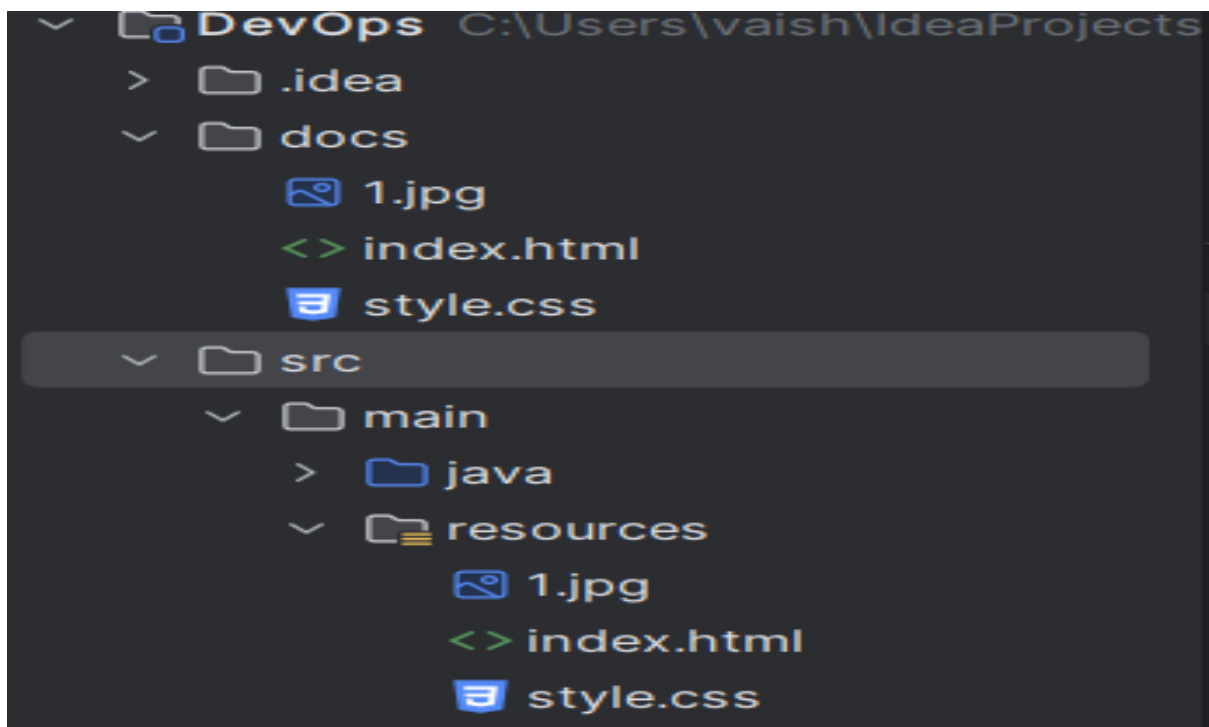
</build>

In this configuration:

- The maven-resources-plugin copies all files from src/main/resources to the /docs folder in the root of your project (not the target directory).
 - This is done during the prepare-package phase, just before Maven prepares the package.
2. **Build the Project:** Run the following Maven command to build your project and copy the resources to the /docs folder:

```
1 mvn clean install
2
```

- After this, your index.html , style.css , and project. logo.png files should now be inside the docs folder in the root of your project.



3. Push Changes to GitHub : Now that the files are in the /docs folder, they are ready to be served by GitHub Pages.

Follow these steps:

- ☐ **Stage the changes (the updated /docs folder):**

```
1 git add docs/*
2 git commit -m "Deploy site to GitHub Pages"
3
```

- ☐ **Push to GitHub:**

```
1 git push origin master # Or the branch you are using, maybe 'main' in some cases
2
```

4. Enable GitHub Pages: After pushing to the main branch, follow these steps to enable GitHub Pages:

- Go to your GitHub repository.
- Navigate to Settings > Pages (on the left sidebar).
- Under the Source section, select the main branch and /docs folder as the source.
- Click Save.

5. Access your website: Your static website is now hosted on GitHub Pages! You can access and copy the link:

```
1 https://<your-github-username>.github.io/<your-repository-name>/
2
```

6. Write a Simple Selenium Test with TestNG

- Create a package org.test and new Java class WebPageTest.java in the src/test/java directory.

Example of a simple TestNG test using Selenium:

```
package org.test;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;

import static org.testng.Assert.assertTrue;

public class WebpageTest {
    private static WebDriver driver;

    @BeforeTest
    public void openBrowser() throws InterruptedException {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        Thread.sleep(2000);
    }
}
```

```

        driver.get("https://raushankumarbaitha.github.io/VTU-DEV-MVN/");
    }

    @Test
    public void titleValidationTest(){
        String actualTitle = driver.getTitle();
        System.out.println(actualTitle);
        String expectedTitle = "Webpage Design";
        Assert.assertEquals(actualTitle, expectedTitle);
        assertTrue(true, "Title contains 'Design'");
    }

    @AfterTest
    public void closeBrowser() throws InterruptedException {
        Thread.sleep(1000);
        driver.quit();
    }
}

```

8. Run the Test:

- In IntelliJ, right-click the WebPageTest class and select Run 'WebPageTest'.
- The test will launch Chrome, open the webpage, and validate the title.
- **Use of maven-jar-plugin to package the project into a jar file. | | Showing how to run a main class and show simple output:**
- **Steps to Package the Project as a JAR and Run a Main Class:**

1. Add maven-jar-plugin to pom.xml (Go to browser ->search for maven jar plugin 3.2.0)

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.2.0</version>
  <configuration>
    <archive>
      <manifestEntries>
        <Main-Class>org.example.Main</Main-Class>
      </manifestEntries>
    </archive>
  </configuration>
</plugin>

```

- This will tell Maven to include the Main-Class in the JAR manifest and specify the main class that should be executed when the JAR is run.

2. Create a Main Class:

In your `src/main/java` directory, create a class with a main method. For example, create a `MainClass.java` under `org.example` :

Example:

```
package org.example;

public class MainClass {

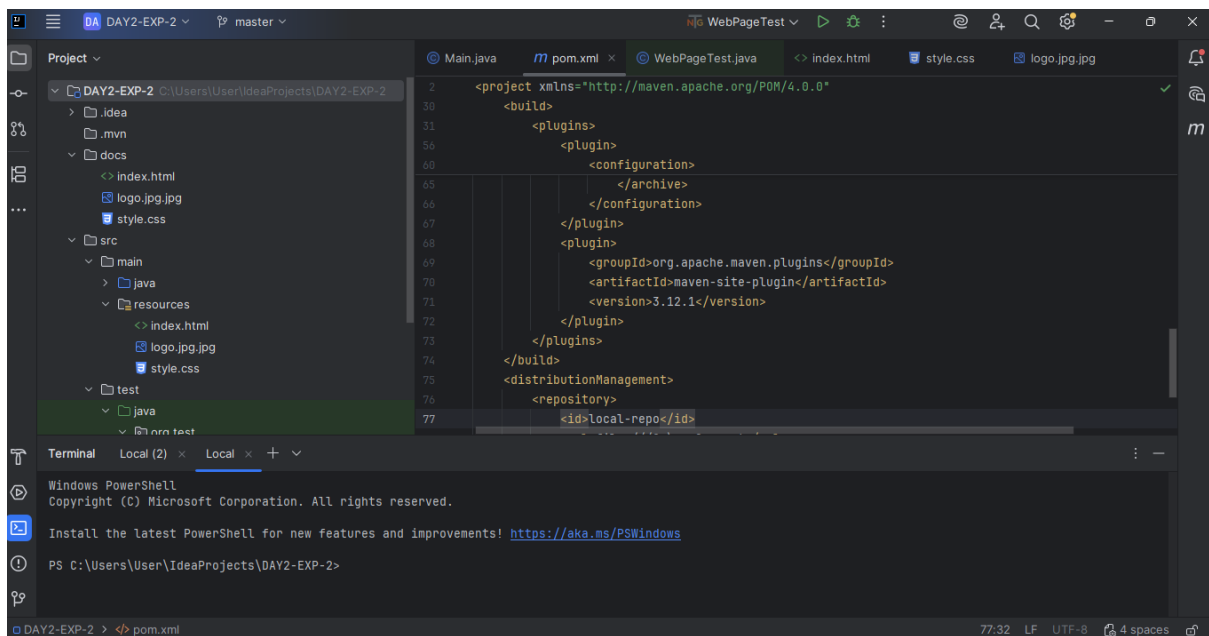
    public static void main(String[] args) {

        System.out.println("Hello, this is a simple output from the main class!");

    }

}
```

- This class contains a simple main method that prints output when run.



3. Package the Project into a JAR:

- After configuring the plugin and creating the `MainClass` , run the following Maven command to build the project and package it into a JAR file:

```
1 mvn clean package
2
```

- This will clean any previous builds, compile the source code, and package it into a JAR file located in the `target` directory (e.g., `target/your-project-name.jar`).

- **4. Run the JAR File:**

- Once the JAR is created, you can run it with the following command:
- This will execute the main method from your `MainClass` and print the message:

- **Output=Hello, this is a simple output from the main class!**

Maven Site and Deploy Commands

1. mvn site Command:

The mvn site command is used to generate a project website containing reports like dependencies, build details, test results, and more.

Steps to Use mvn site:

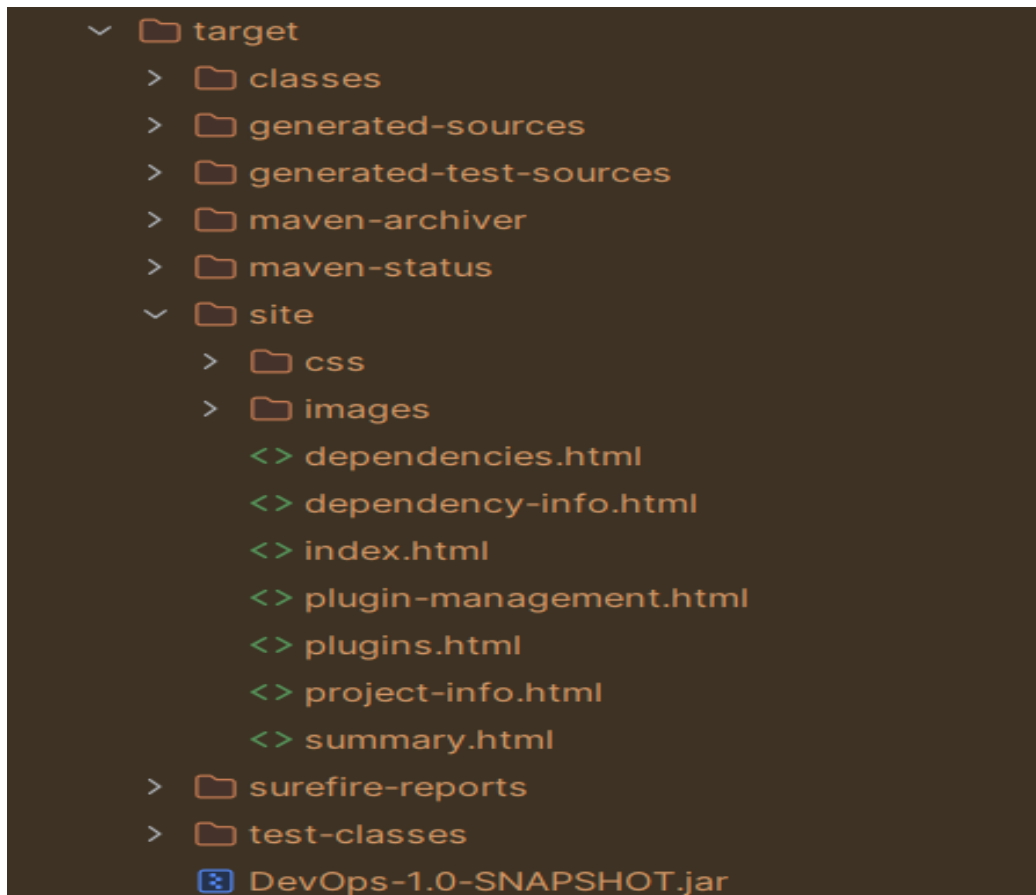
Step 1 : Add Site Plugin in pom.xml

- Before running the site command, you need to add the Maven Site Plugin inside the pom.xml :

```
<build>
<plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-site-plugin</artifactId>
    <version>3.12.1</version> <!-- Use latest version -->
  </plugin>
</plugins>
</build>
```

Step 2: Run the Site Command

```
1 mvn site
2
```



mvn deploy Command:

The mvn deploy command is used to upload the built artifact (JAR, POM, etc.) to a repository for distribution and sharing.

Steps to Use mvn deploy:

- Since you don't have a remote repository, we will configure a local repository.

Step 1: Create a Local Repository

```
1 mkdir D:\my-local-maven-repo
2
```

Step 2: Configure pom.xml for Local Deployment

Add the following inside in pom.xml after build:

```
<distributionManagement>
```

```
  <repository>
```

```
    <id>local-repo</id>
```

```
<url>file:///C:/my-local-maven-repo</url>
```

```
</repository>
```

```
</distributionManagement>
```

Step 3: Run the Deploy Command

```
1 mvn deploy
2
```

- ✓ Maven builds the project.
- ✓ Stores the artifact (JAR, POM, etc.) in D:/my-local-maven-repo .

Step 4: Verify Deployment

Navigate to D:/my-local-maven-repo/ and check if the project is stored correctly:

```
D:/my-local-maven-repo/
├─ org/example/CA-MVN/1.0-SNAPSHOT/
|   ├─ CA-MVN-1.0-SNAPSHOT.jar
|   └─ CA-MVN-1.0-SNAPSHOT.pom
```

Experiment 3: Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation

1. Introduction to Gradle

- **Gradle** is a modern build automation tool designed to be highly flexible, fast, and scalable. It is widely used in Java projects, Android development, and many multi-language projects. Here's what makes Gradle stand out:
- **Flexible Build Scripts:** Gradle uses a Domain Specific Language (DSL) based on either Groovy (by default) or Kotlin. This provides a more dynamic and expressive way to define build logic compared to static XML configurations (as used in Maven).
- **Incremental Builds:** Gradle optimizes build times by determining what parts of the project have changed and rebuilding only those parts.
- **Task Automation:** Everything in Gradle is treated as a task, allowing you to create custom tasks or reuse existing ones for compiling code, running tests, packaging, and more.
- **Dependency Management:** Like Maven, Gradle can automatically download and manage dependencies from remote repositories (e.g., Maven Central, JCenter, or custom repositories).

2. Understanding Gradle Build Scripts

- A **build script** in Gradle is a programmatic file that instructs Gradle on how to build your project. It can be written in two main DSLs:
- ☐ **Groovy DSL (build.gradle):** The traditional and most common syntax.
- ☐ **Kotlin DSL (build.gradle.kts):** A statically-typed alternative that leverages Kotlin's language features.

Explanation of Components:

- **build.gradle:**

This is the main build script written in Groovy (or Kotlin if you choose). It defines plugins, repositories, dependencies, and tasks.

- **settings.gradle:**

A small script that defines the project's name and, in multi-project builds, the included subprojects.

- **gradlew / gradlew.bat:** The Gradle wrapper scripts. They allow you to run Gradle without requiring a separate installation on every machine by automatically downloading the correct Gradle version.

- **src/main/java:**

Contains your application's source code.

- **src/test/java:**

Contains your unit tests.

Explanation:

plugins block: Declares the plugins used. The `java` plugin adds Java compilation and testing tasks, while the `application` plugin adds tasks for running the application.

group/version: Sets project coordinates.

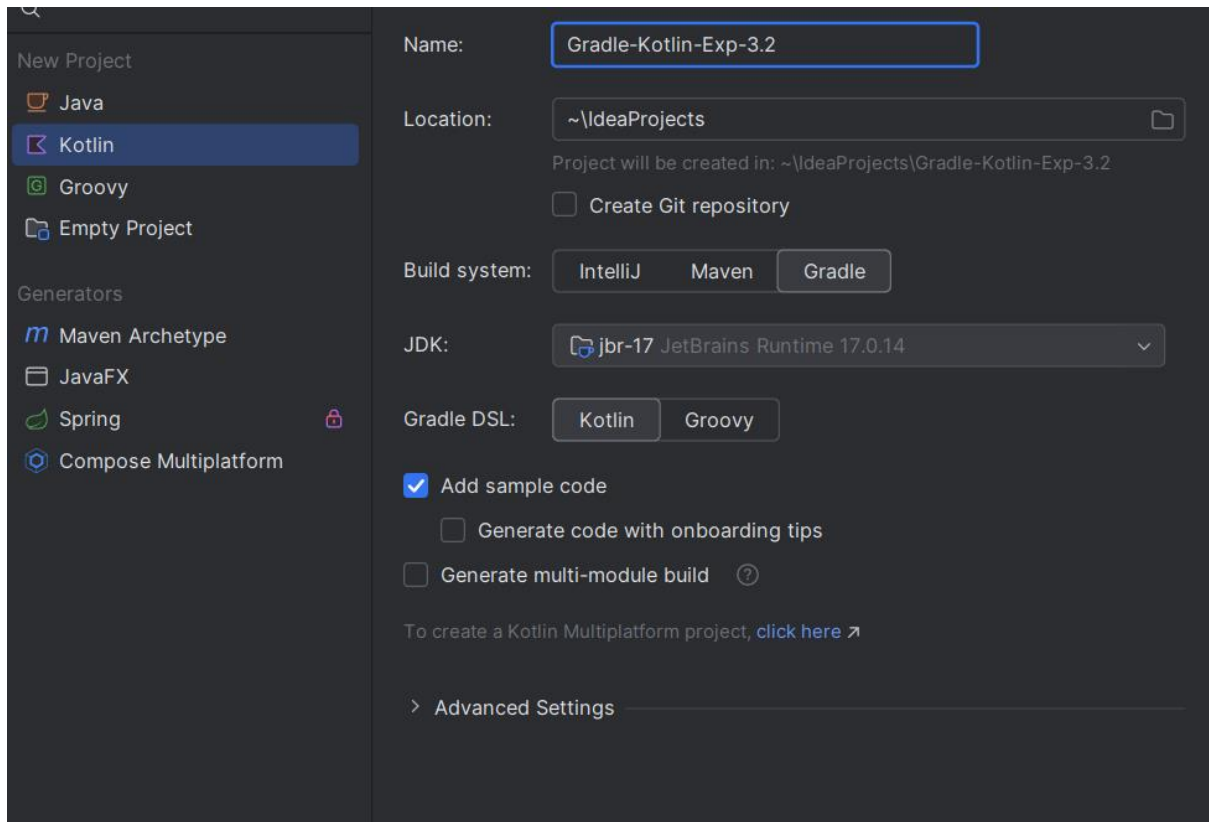
repositories block: Configures the repository (Maven Central) where dependencies will be resolved.

dependencies block: Lists the libraries your project depends on.

application block: Specifies the main class of your application.

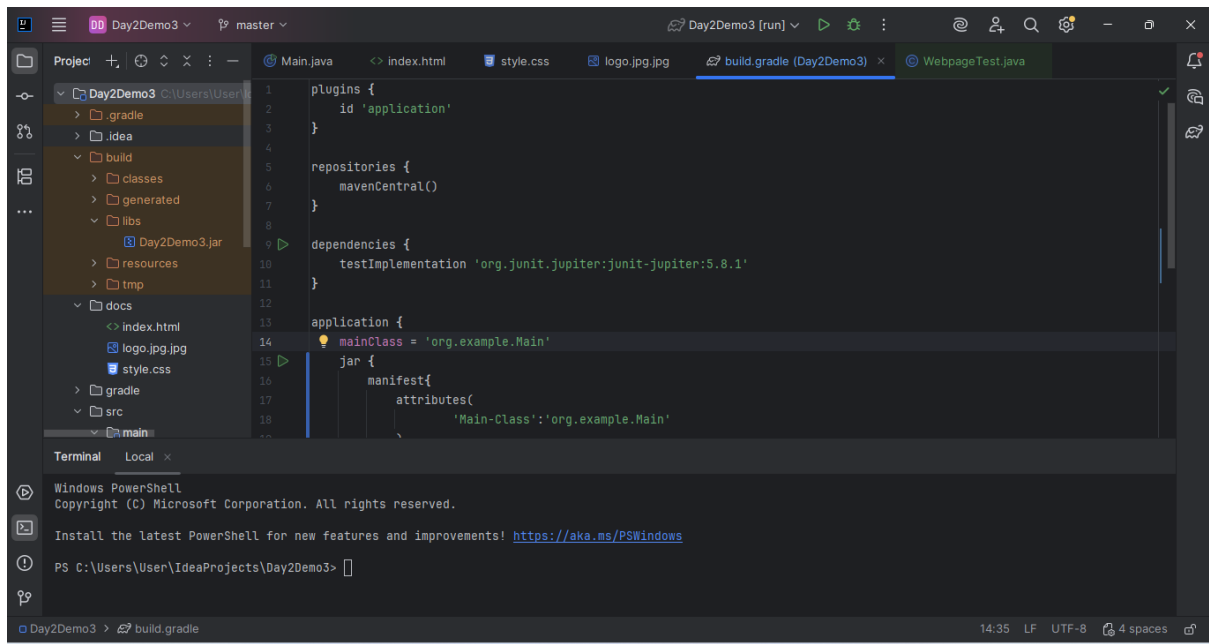
Custom task: Defines a task named `hello` that prints a greeting when run.

3.1 Groovy DSL Example (build.gradle)



STEPS:

1. Click on "New Project".
2. Select "Gradle".
3. **Choose Groovy DSL** (Domain Specific Language) for the build script.
4. Click Finish.



- **Step 1: Modify build.gradle (Groovy DSL)**

```
plugins {  
    id 'application'  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'  
}  
  
application {  
    mainClass = 'org.example.Main'  
}
```

- **Step 2: Create Main.java in src/main/java/org/example**

```
package org.example;  
  
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

- **Step 3: Build and Run the Project**

In IntelliJ IDEA, open the Gradle tool window (View → Tool Windows → Gradle).

Click Tasks > application > run .

Or

run from terminal:

```
gradle run
```

Create one directory named as website inside resources and paste the html css and image inside website.

// Task to copy HTML, CSS, and images to 'docs' folder

```
tasks.register('prepareDocs', Copy) {
    description = 'Copies HTML, CSS, and images to the docs folder.'
    group = 'documentation'

    from('src/main/resources/website') {
        include '**/*.html', '**/*.css', '**/*.js', '**/*.png', '**/*.jpg', '**/*.jpeg', '**/*.gif', '**/*.svg'
    }
    into("$rootDir/docs")
}
```

Run the following command in your terminal:

```
./gradlew prepareDocs
```

Step 3: Commit and push to github

- 1.git add .
2. git commit -m "first commit"
3. git remote add origin <url>
4. git push origin master

Step 4:Enable Github Pages

- 1.Go to github repo->settings->pages.
- 2.Select the /docs folder as the source.

- **Your website will be hosted at github pages:**

Testing the website using selenium java and test NG

Step1: Add selenium & tesNG dependencies in build.gradle

```
<dependencies {
    implementation 'org.seleniumhq.selenium:selenium-java:4.31.0'
    testImplementation 'org.testng:testng:7.11.0'
}
```

```
test{
    useTestNG()
}
```

Step2: Write a Test Script(src/test/java/org/test/WebpageTest.java)

```
package org.test;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;

import static org.testng.Assert.assertTrue;

public class Webpagetest {
    private static WebDriver driver;

    @BeforeTest
    public void openBrowser() throws InterruptedException {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        Thread.sleep(2000);
        driver.get("https://raushankumarbaitha.github.io/gradle/");
    }

    @Test
    public void titleValidationTest(){
        String actualTitle = driver.getTitle();
        System.out.println(actualTitle);
        String expectedTitle = "My Simple Website";
        Assert.assertEquals(actualTitle, expectedTitle);
        assertTrue(true, "Title contains 'Design'");
    }

    @AfterTest

    public void closeBrowser() throws InterruptedException {
        Thread.sleep(1000);
        driver.quit();
    }
}
```

Step 3:Run the Tests

- Open the gradle tool->verification->test

gradle test // fails sometimes due to terminal issue

Packaging a gradle project as a jar

Step1: Modify build.gradle for java packaging

```
plugins {  
    id 'application'  
    id 'java'  
}  
  
jar {  
    manifest {  
        attributes(  
            'Main-Class': 'org.example.Main'  
        )  
    }  
}
```

Step2: Build and package the jar

gradle jar

- The jar file will be generated in build/libs.

Step3: Run the Jar

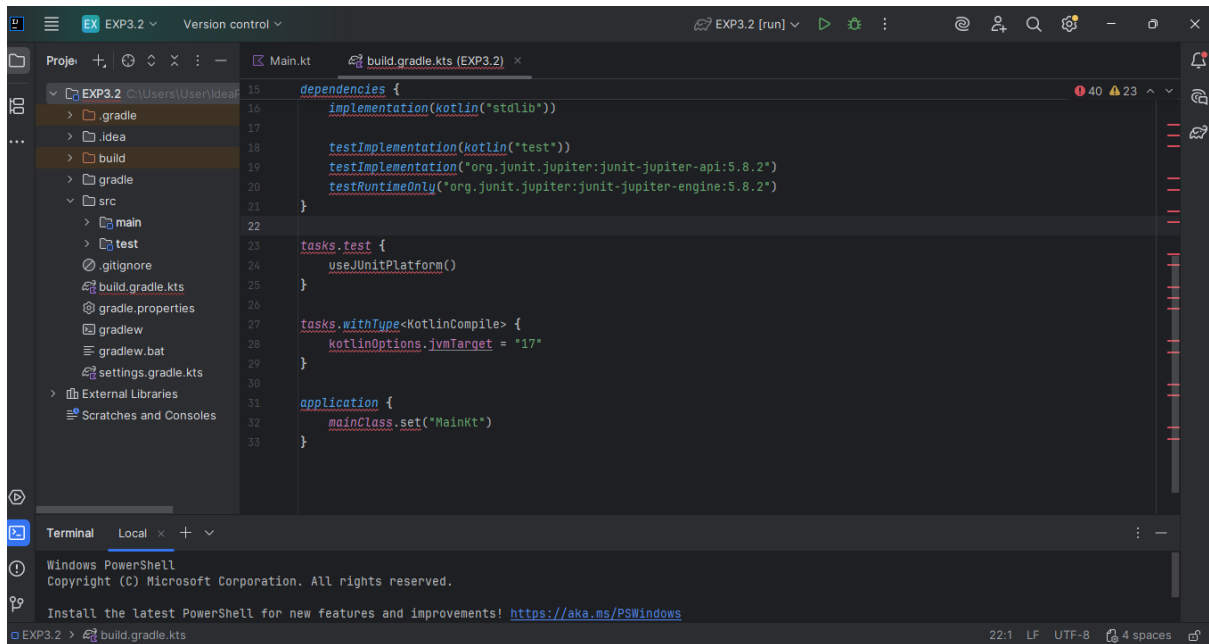
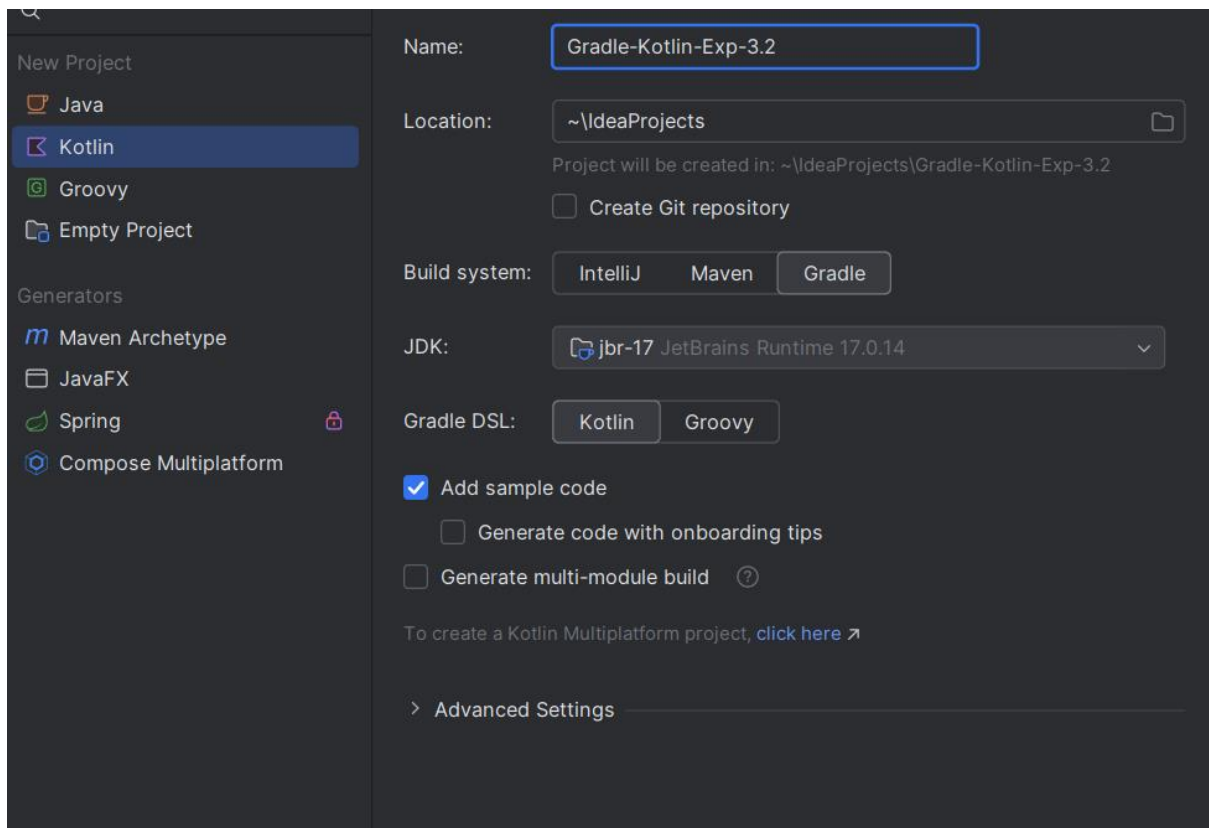
java -jar .\build\libs\<project-name.jar>

3.2 Kotlin DSL Example (build.gradle.kts)

Setting Up the Gradle Project

Step 1: Create a New Project \

1. Open IntelliJ IDEA.
2. Click on File > New > Project.
3. Select Gradle as the build system.
4. Choose Kotlin as the language.
5. **Select Gradle Kotlin DSL (it will generate**
6. **Name your project (e.g., build.gradle.kts). MVNGRDKOTLINDEMO).**
7. Set the JDK (use JDK 17.0.4, since that's your version).
8. Click Finish.



- Modify build.gradle.kts

import org.jetbrains.kotlin.gradle.tasks.KotlinCompile

```
plugins {  
    kotlin("jvm") version "1.8.21"  
    application  
}
```

```

group = "org.example"
version = "1.0-SNAPSHOT"

repositories {
    mavenCentral()
}

dependencies {
    testImplementation(kotlin("test"))
    testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.2")
}

tasks.test {
    useJUnitPlatform() // you can also use TestNG
}

tasks.withType<KotlinCompile> {
    kotlinOptions.jvmTarget = "17" // Match with your JDK version
}

application {
    mainClass.set("MainKt") // Update this if using a package
}

```

3.Creating the Main Kotlin File

```

fun main(args: Array<String>) {
    println("Hello World!")
    println("This is Raushan")
    println("Welcome to the world of Kotlin")
    println("I am learning Kotlin")
}

```

1.Run this code

- **Building and Running the Project**

1.To build

```
./gradlew build
```

2.To run

```
./gradlew run
```

- Packaging as a JAR

Step 1: Create a Fat (Uber) JAR

- **Modify build.gradle.kts :**

```

tasks.register<Jar>("fatJar") {
    group = "build"
}

```

```

description = "Creates a fat JAR including all dependencies"
archiveClassifier.set("all")
duplicatesStrategy = DuplicatesStrategy.EXCLUDE
manifest {
    attributes["Main-Class"] = "MainKt"
}
from(sourceSets.main.get().output)
from({
    configurations.runtimeClasspath.get().map {
        if (it.isDirectory) it else zipTree(it)
    }
})
with(tasks.jar.get() as CopySpec)
}

```

Step 2. Build fat Jar

`./gradlew fatJar`

Step 3. Run the fat Jar

`java -jar ./build/libs/project-path-name`

Or

`java -jar ./build/libs/Gradle-Kotlin-Exp-3.2-1.0-SNAPSHOT-all.jar`

Adding Custom Gradle Tasks

Example: A simple task that prints "Hello, Gradle!"

- you can add this code anywhere in `build.gradle.kts`

```

tasks.register("hello") {
    doLast {
        println("Hello, Gradle!")
    }
}

```

Experiment 4: Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle

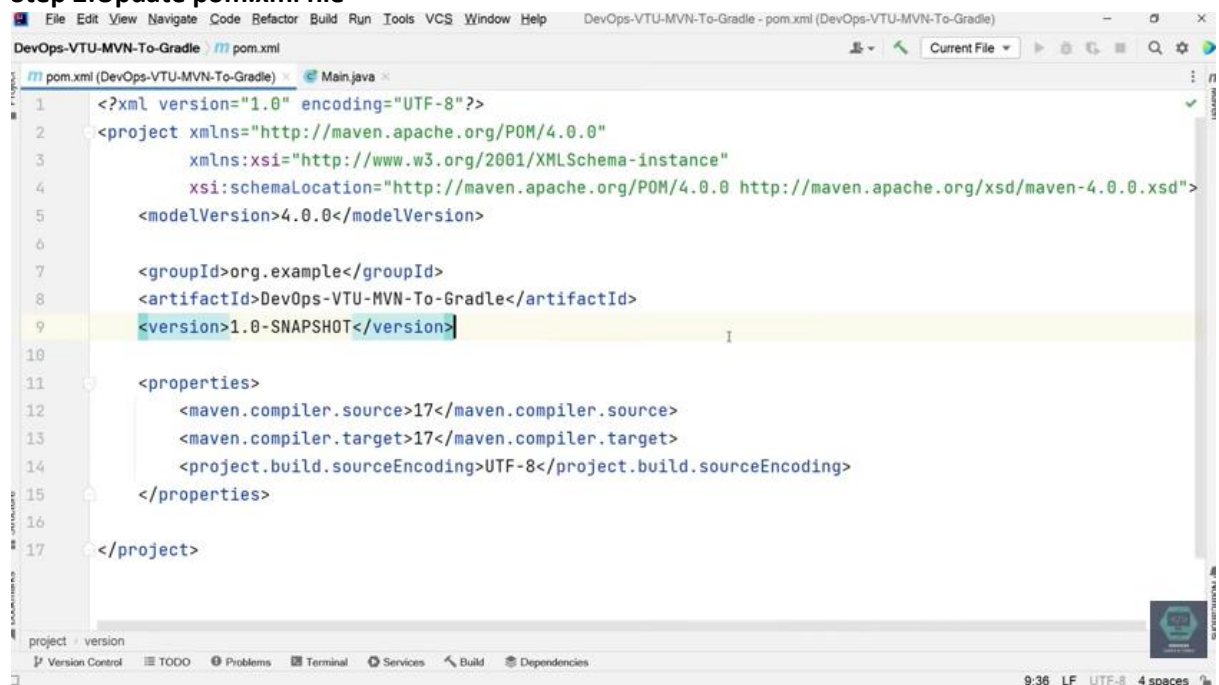
Part A: Build and Run a Java Application with Maven

Create and Build a Java Application with Maven:

Step 1: Create a Maven Project in IntelliJ IDEA\

1. Open IntelliJ IDEA Launch IntelliJ IDEA and click on File → New → Project.
2. Select language as java.
3. Select Maven as build system
4. Select jdk version 17.
5. Create

Step 2: Update pom.xml file



1. Add apache maven compiler plugins

Create one build tag just after the properties → plugins tag inside that add the plugin

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.11.0</version>

<configuration>

<source>17</source>

<target>17</target>

</configuration>

</plugin>

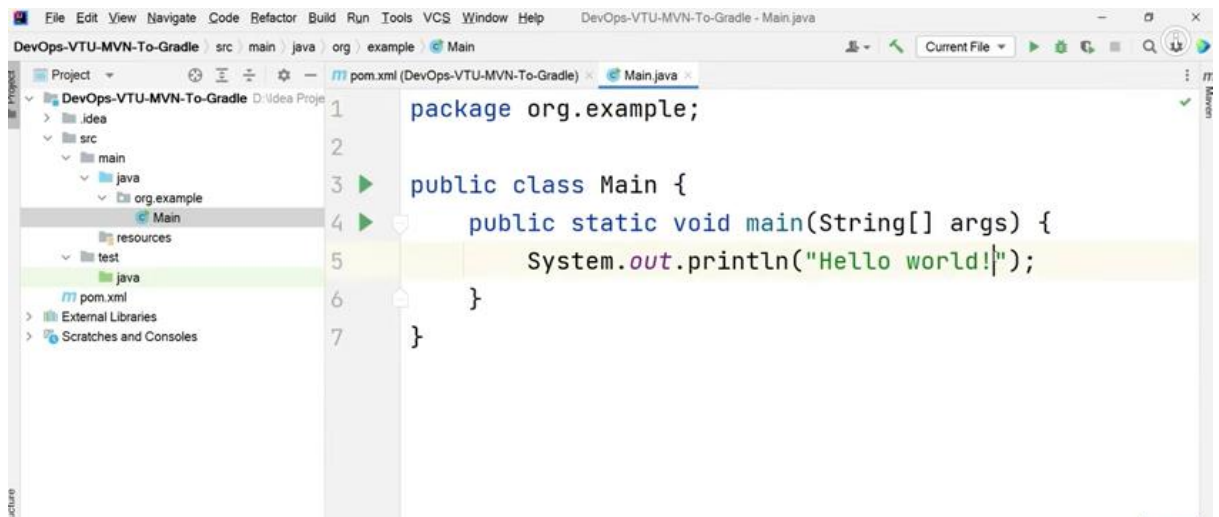
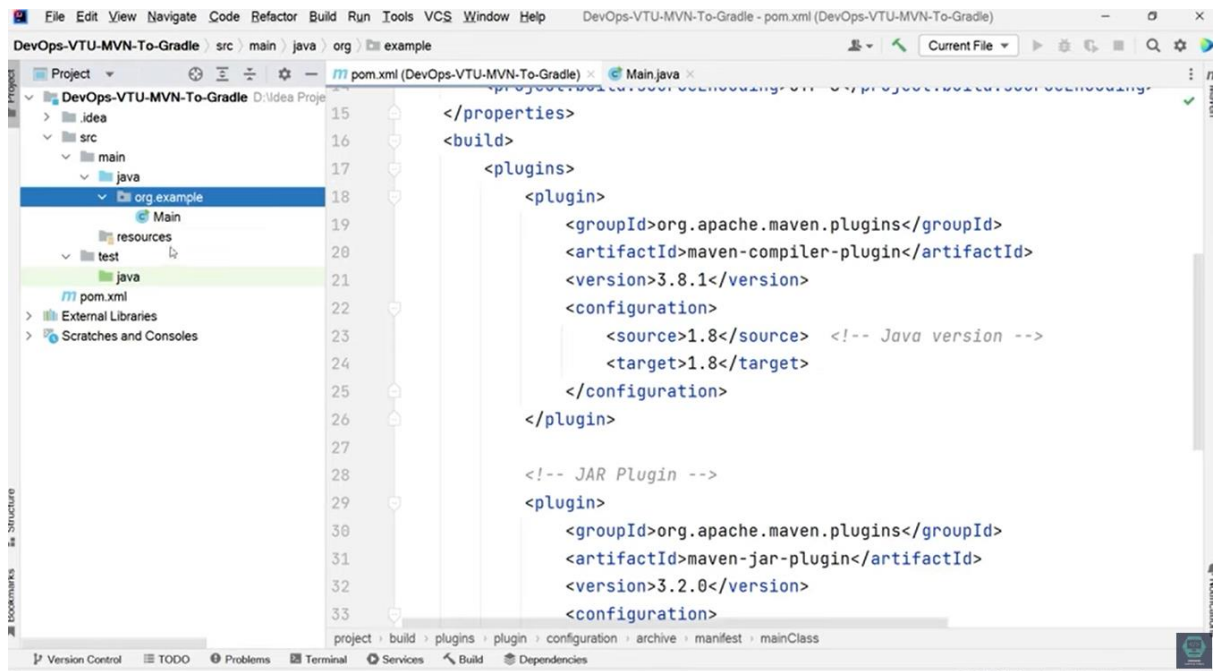
2. Add maven jar plugin 3.3.0

<plugin>

<groupId>org.apache.maven.plugins</groupId>

```
<artifactId>maven-jar-plugin</artifactId>
<version>3.3.0</version>
<configuration>
  <archive>
    <manifest>
      <addClasspath>true</addClasspath>
      <mainClass>org.example.Main</mainClass>
    </manifest>
  </archive>
</configuration>
</plugin>
```

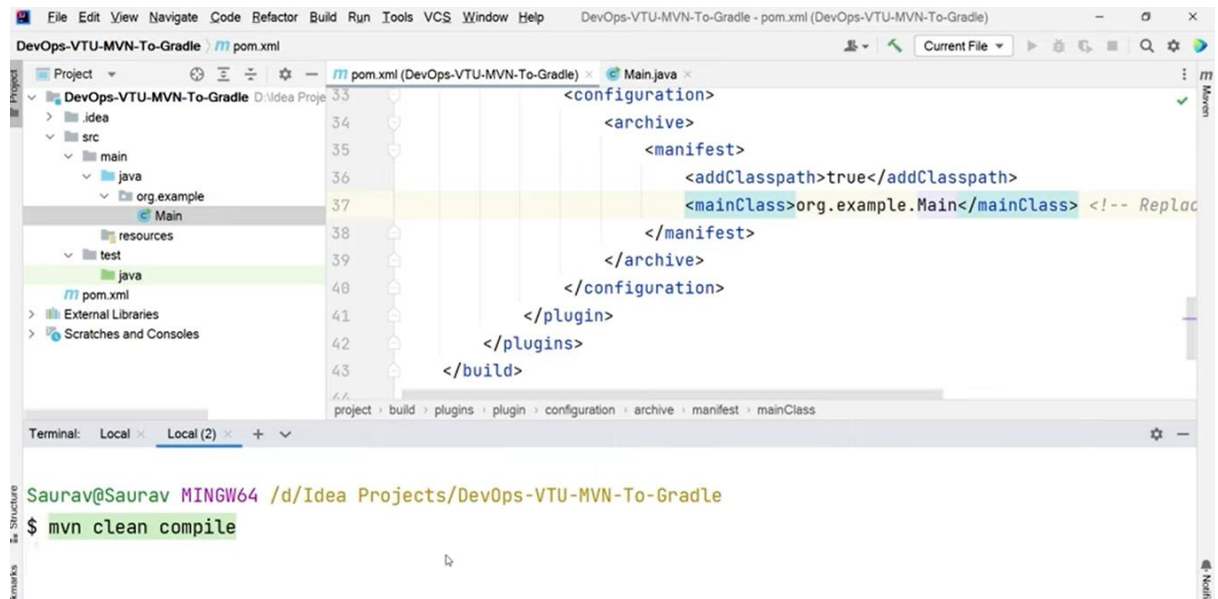
```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.2.0</version>
  <configuration>
    <archive>
      <manifest>
        <addClasspath>true</addClasspath>
        <mainClass>org.example.Main</mainClass>
      </manifest>
    </archive>
  </configuration>
</plugin>
```



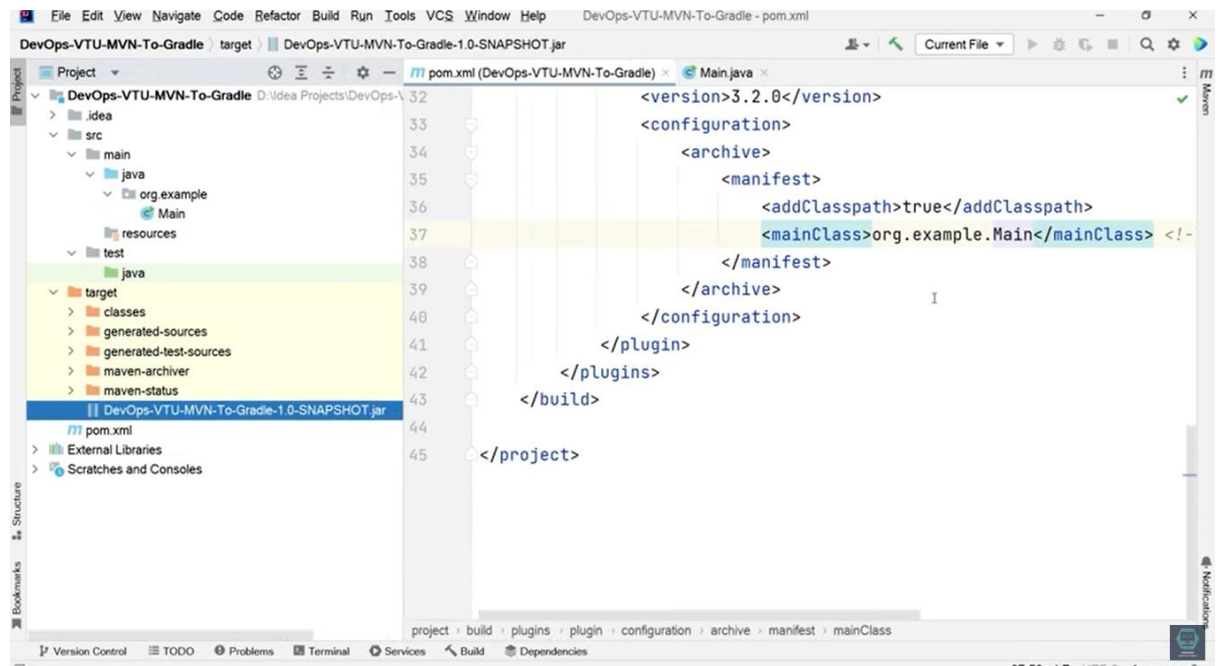
Step 3: Build and Run the Maven Project

1.Run the command

mvn clean compile



mvn package



→it will create one jar file inside target

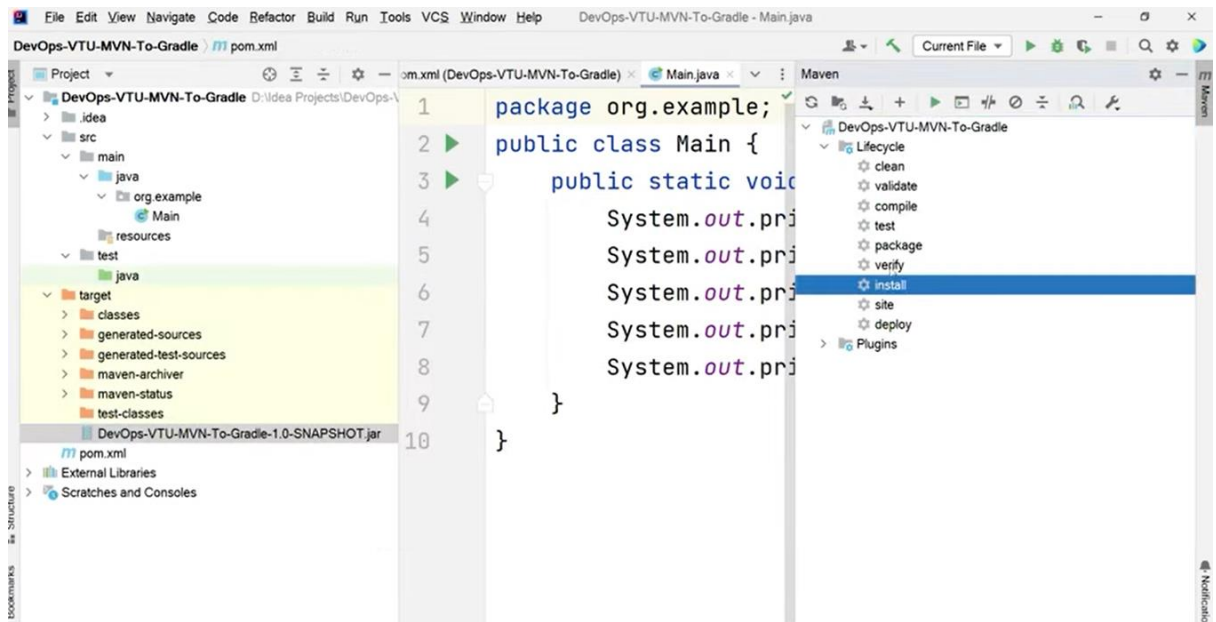
2.Run the Jar file

```
java -jar .\target\MVN-To-Gradle-1.0-SNAPSHOT.jar
```

Output: Hello world

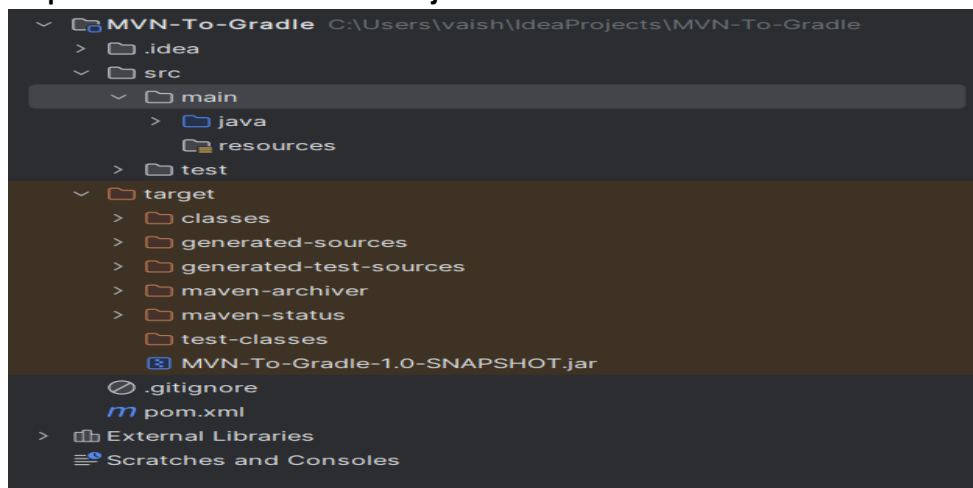
- To check if there any dependency need to install ,then it will install.

mvn clean install



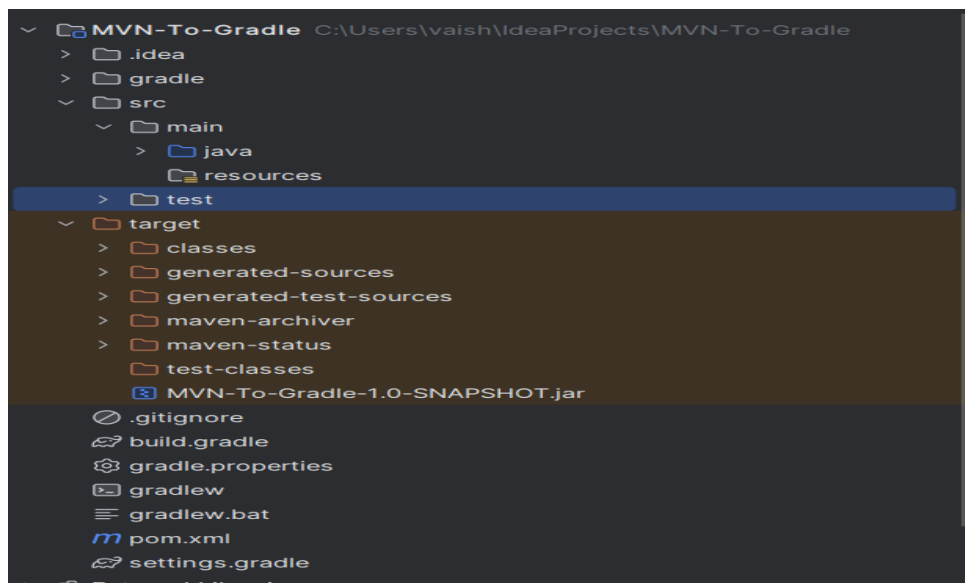
Migrate Maven Project to Gradle

Step 1: Initialize Gradle in Your Project



gradle init --type pom

This command will convert your Maven pom.xml into a Gradle build.gradle file.



Step 2: Go to build.gradle and modify plugins

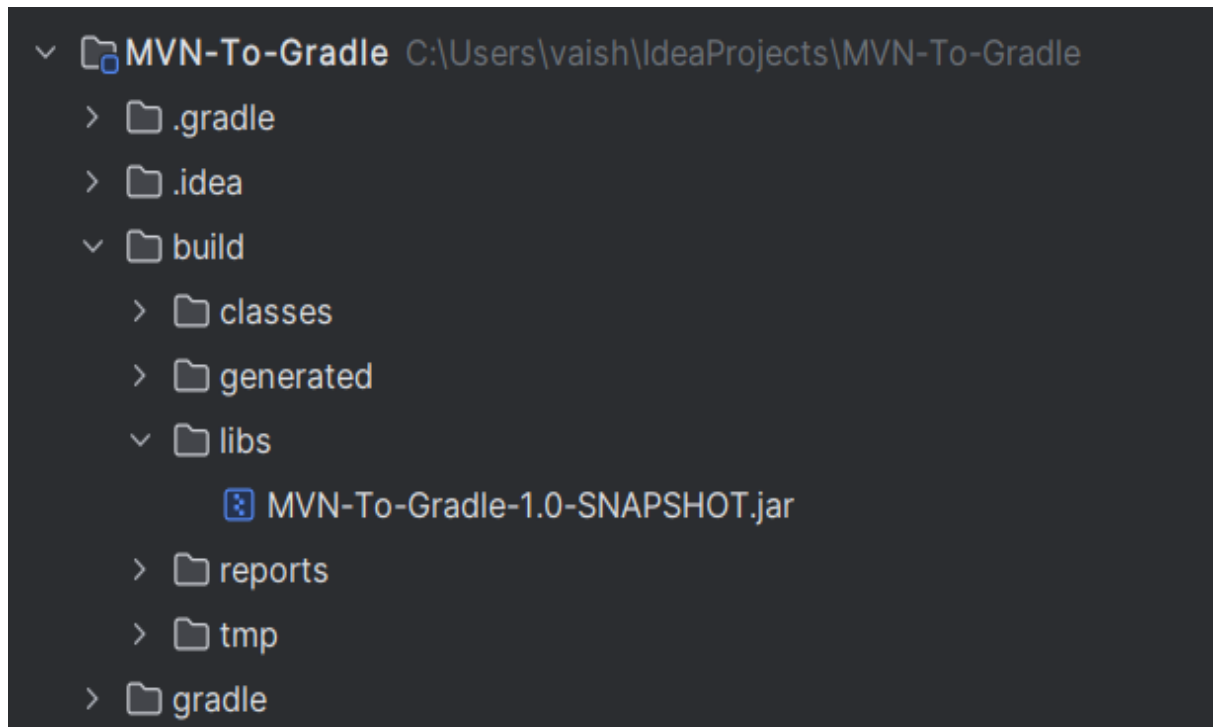
```
plugins {  
    id 'java-library'  
    id 'maven-publish'  
    id 'java'  
}
```

- **Add j-unit dependencies 4.13.2 and jar file**

```
dependencies {  
    testImplementation 'junit:junit:4.13.2'  
}  
  
jar {  
    manifest {  
        attributes(  
            'Main-Class': 'org.example.Main'  
        )  
    }  
}
```

Step 3: Build and Run the Gradle Project

gradle clean build



Step4:

```
java -jar .\build\libs\MVN-To-Gradle-1.0-SNAPSHOT.jar
```

output : Hello world

Experiment 5: Introduction to Jenkins: What is Jenkins?, Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use

What is Jenkins?

Jenkins is an **open-source automation server** used for:

Continuous Integration (CI) – Automatically testing and integrating code changes

Continuous Deployment (CD) – Automating application deployment

Building Pipelines – Managing end-to-end software development workflows

Plugin-Based Extensibility – Supporting tools like Maven, Gradle, Ansible, Docker, and Azure DevOps

Why Use Jenkins?

Automates builds and tests

Reduces manual intervention

Improves software quality

Works with multiple tools and platforms

Installing Jenkins

Jenkins can be installed using multiple methods:

Windows Installer (.msi) – Recommended for Windows

Linux Package Manager – Best for Linux Users

jenkins WAR File – Universal method using Java

Prerequisites

Java 11 or 17 is required for Jenkins.

Check Java version:

If Java is not installed, download it from:

Installing Jenkins on Windows

Step 1: Download Jenkins

Download from: [Download and deploy](#)

Choose **Windows Installer (.msi)** for an easy setup.

Step 2: Install Jenkins

Run the downloaded .msi file.

Follow the installation wizard.

Select **Run Jenkins as a Windows Service** (recommended).

Choose the **installation directory** (default: C:\Program Files\Jenkins).

Click **Install** and wait for the setup to complete

[illegible]

Step 3: Start Jenkins

Open **Services** (`services.msc`) and ensure **Jenkins** is running

Open a web browser and go to:

<http://localhost:8080>

Step 4: Unlock Jenkins

Find the initial **Admin Password** in:

C:\Program Files\Jenkins\secrets\initialAdminPassword

Copy the password and paste it into the Jenkins setup page

```
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\BIET>

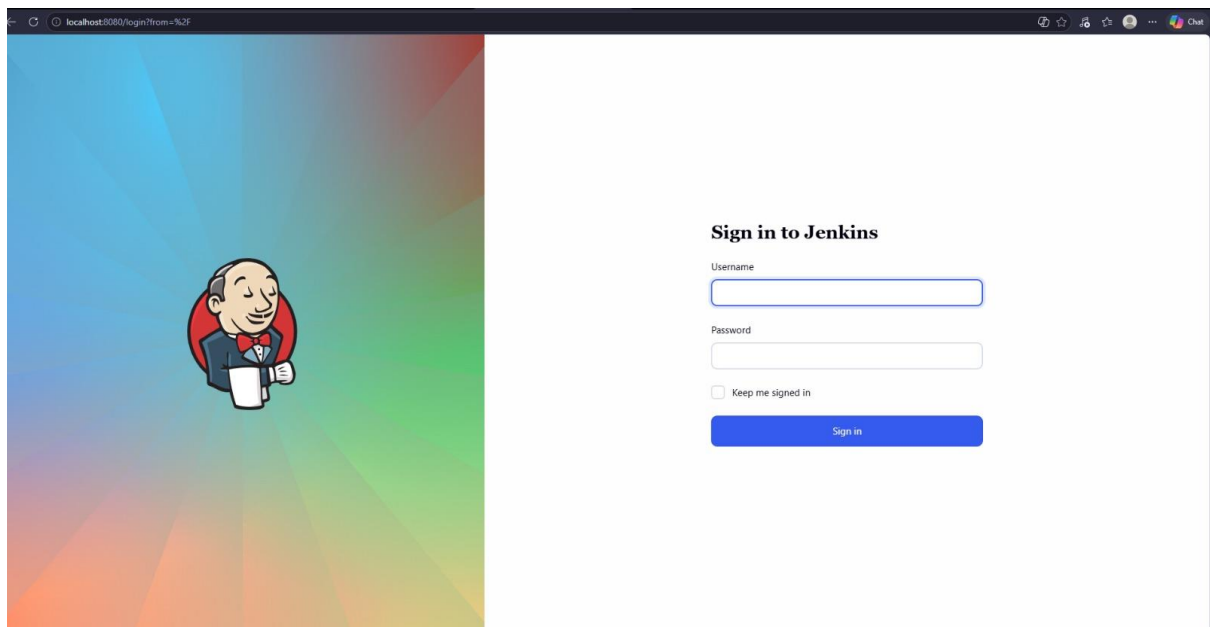
E:\>cd Janlins
The system cannot find the path specified.

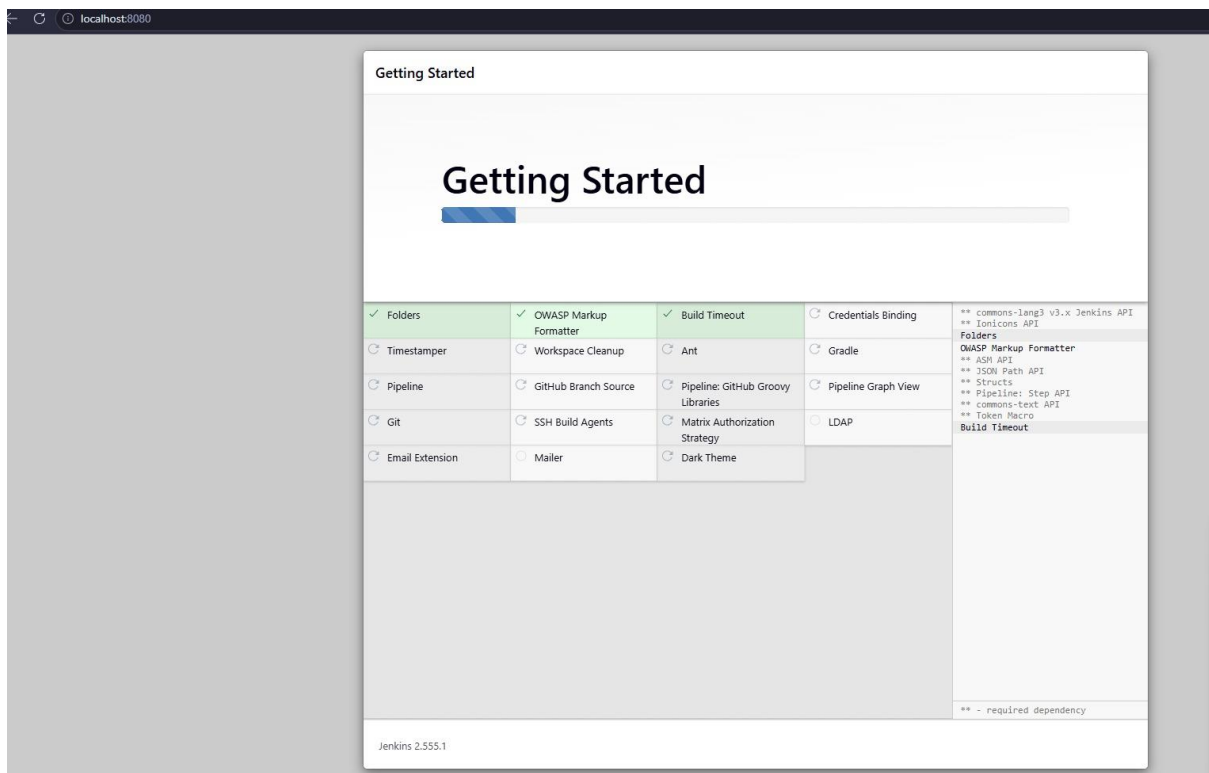
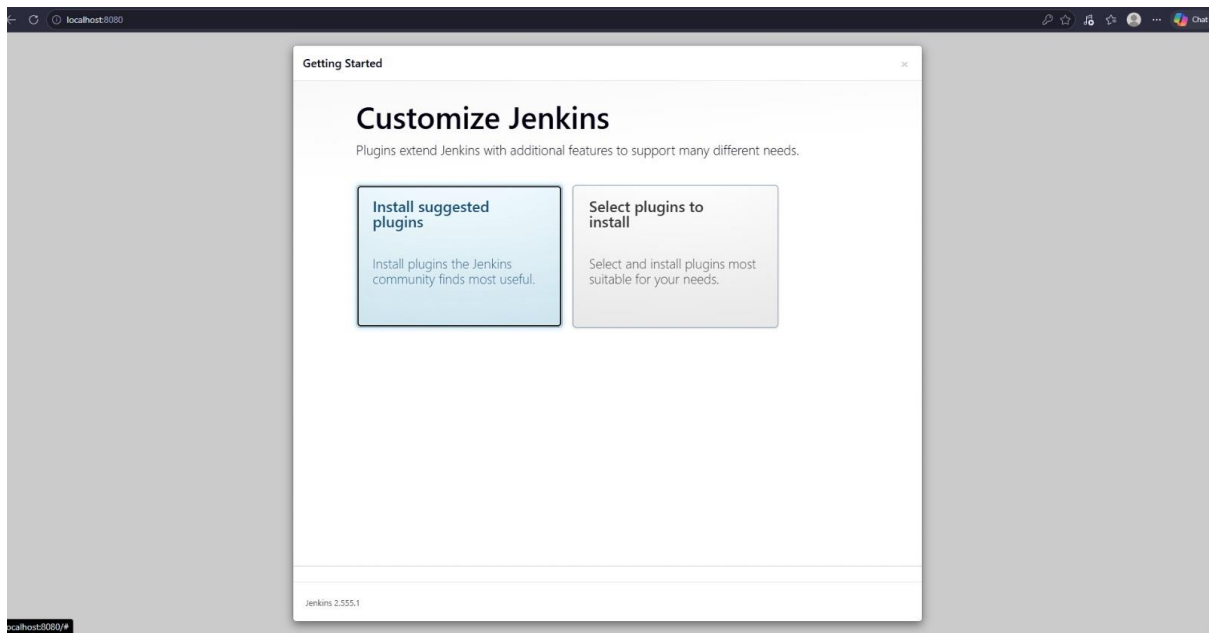
E:\>cd Jenkins

E:\Jenkins>java -jar jenkins
Error: Unable to access jarfile jenkins

E:\Jenkins>java -jar jenkins.war
Running from: E:\Jenkins\jenkins.war
webroot: C:\Users\BIET\jenkins\war
2026-04-16 09:44:24.227+0000 [id=1] INFO winstone.Logger$logInternal: Beginning extraction from war file
2026-04-16 09:44:24.921+0000 [id=1] WARNING o.e.j.ee9.nested.ContextHandler$setContextPath: Empty contextPath
2026-04-16 09:44:24.961+0000 [id=1] INFO org.eclipse.jetty.server.Server$doStart: jetty-12.1.6; built: 2026-01-27T18:53:19.102Z; git: 88ca559672b1c8858b3c568bb0293fad4e5e90f; jvm 21.0.10+0-LTS-217
2026-04-16 09:44:25.263+0000 [id=1] INFO o.e.j.e.s.StandardDescriptorProcessor$visitServlet: NO JSP Support for /, did not find org.eclipse.jetty.ee9.jsp.JettyJspServlet
2026-04-16 09:44:25.272+0000 [id=1] INFO o.e.j.s.DefaultSessionIdManager$doStart: Session workerName=node0
2026-04-16 09:44:25.464+0000 [id=1] INFO hudson.WebAppMain$contextInitialized: Jenkins home directory: C:\Users\BIET\jenkins found at: $user.home/.jenkins
2026-04-16 09:44:25.510+0000 [id=1] INFO o.e.j.s.handler.ContextHandler$doStart: Started oeje9n.ContextHandler$CoreContextHandler@369c9bb[Jenkins v2.555.1,,b=file:///C:/Users/BIET/.jenkins/war/,a=AVAILABLE,hmoeje9n.ContextHandler$CoreContextHandler$CoreContextHandler$76d85cc9[STARTED]]
2026-04-16 09:44:25.530+0000 [id=1] INFO o.e.j.server.AbstractConnector$doStart: Started oejs.ServerConnector@7fe3ad9e[HTTP/1.1, (http/1.1)][0.0.0.0:8080]
2026-04-16 09:44:25.530+0000 [id=1] INFO org.eclipse.jetty.server.Server$doStart: Started oejs.Server@1f387978[STARTING][12.1.6,sto=0] @580ms
2026-04-16 09:44:25.531+0000 [id=67] INFO winstone.Logger$logInternal: Winstone Servlet Engine running: controlPort=disabled
2026-04-16 09:44:25.581+0000 [id=87] INFO jenkins.model.Jenkins$init: Starting version 2.555.1
2026-04-16 09:44:25.628+0000 [id=76] INFO jenkins.InitReactorRunner$1$onAttained: Started initialization
2026-04-16 09:44:25.636+0000 [id=96] INFO jenkins.InitReactorRunner$1$onAttained: Listed all plugins
2026-04-16 09:44:25.923+0000 [id=103] INFO jenkins.InitReactorRunner$1$onAttained: Prepared all plugins
2026-04-16 09:44:26.030+0000 [id=114] INFO jenkins.InitReactorRunner$1$onAttained: Started all plugins
2026-04-16 09:44:26.030+0000 [id=120] INFO jenkins.InitReactorRunner$1$onAttained: Augmented all extensions
2026-04-16 09:44:26.107+0000 [id=120] INFO jenkins.InitReactorRunner$1$onAttained: System config loaded
2026-04-16 09:44:26.107+0000 [id=60] INFO jenkins.InitReactorRunner$1$onAttained: System config adapted
2026-04-16 09:44:26.106+0000 [id=90] INFO jenkins.InitReactorRunner$1$onAttained: Loaded all jobs
2026-04-16 09:44:26.109+0000 [id=93] INFO jenkins.InitReactorRunner$1$onAttained: Configuration for all jobs updated
2026-04-16 09:44:26.124+0000 [id=134] INFO hudson.util.Retrier$Start: Attempt #1 to do the action check updates server
2026-04-16 09:44:26.295+0000 [id=93] INFO jenkins.install.SetupWizard$init:

[CR LF]> *****
[CR LF]> *****
[CR LF]> *****
[CR LF]> Jenkins initial setup is required. An admin user has been created and a password generated.
[CR LF]> Please use the following password to proceed to installation:
[CR LF]>
[CR LF]> b43e9213365c41609b494eba726af3c2
[CR LF]>
[CR LF]> This may also be found at: C:\Users\BIET\.jenkins\secrets\initialAdminPassword
[CR LF]>
```





Step 5: Install Recommended Plugins

Jenkins will prompt you to install plugins. Click "**Install Suggested Plugins**".

Step 6: Create Admin User

set up a **Username**, **Password**, and **Email**.
Click **Save and Finish**.

Jenkins is now ready!

Access it anytime at:

Jenkins is ready!

Your Jenkins setup is complete.

Start using Jenkins

Jenkins 2.555.1

<http://localhost:8080>

Installing Jenkins Using WAR File

This method allows you to run Jenkins without installing it as a service

Download from: [Download and deploy](#)

Choose **Generic Java Package (.war)**.

Step 2: Run Jenkins Using Java

Navigate to the folder where the .war file is downloaded and run:

◆ This will start Jenkins on port **8080**.

✔ Step 3: Open Jenkins in Browser

Go to:

Step 4: Unlock Jenkins & Setup

Follow the **same steps** as the Windows/Linux installation:

Find the initial password

Install plugins

Create an admin user

◆ **Jenkins is now running without installation!**

To stop Jenkins, press **CTRL + C** in the terminal.

3. Configuring Jenkins for First Use

3.1 Understanding the Jenkins Dashboard

After logging in, you will see:

New Item → Create Jobs/Pipelines

Manage Jenkins → Configure System, Users, and Plugins

Build History → View previous builds

Credentials → Store secure authentication details

3.2 Installing Additional Plugins

Jenkins supports **plugins** for various tools like Maven, Gradle, Docker, and Azure DevOps.

To install a plugin:

Go to **Manage Jenkins** → **Manage Plugins**

Search for the required plugin

Click **Install without Restart**

3.3 Setting Up Global Tool Configuration

Configure Java, Maven, and Gradle in Jenkins:

Go to **Manage Jenkins** → **Global Tool Configuration**

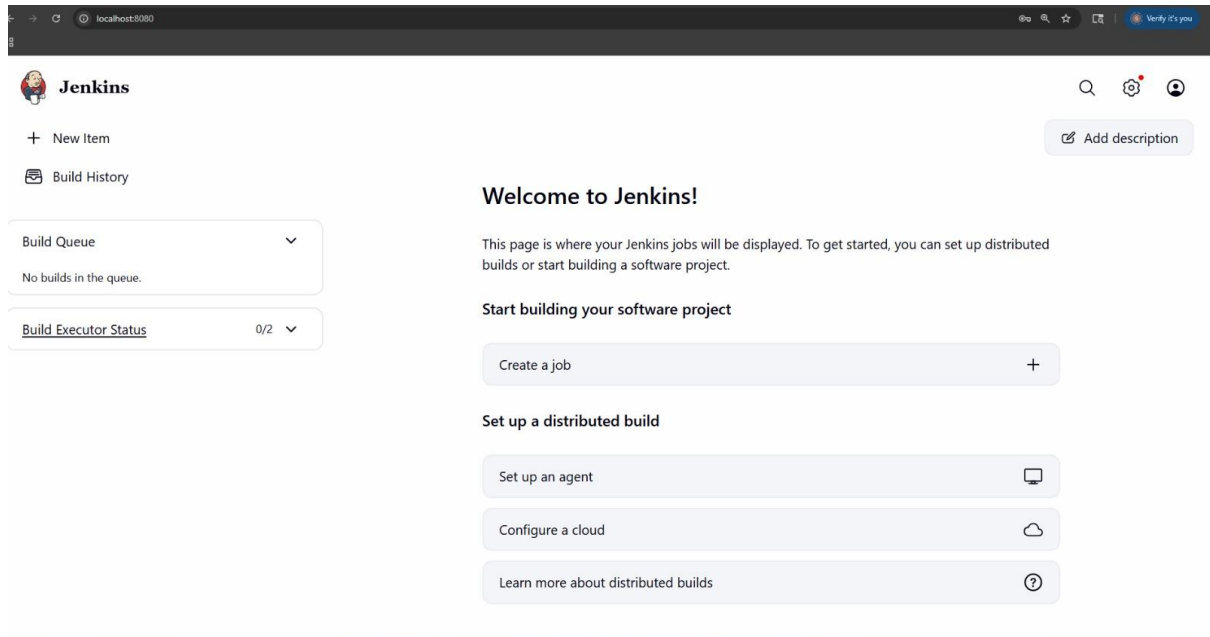
Add paths for:

JDK (C:\Program Files\Java\jdk-17)

Maven (C:\Maven\apache-maven-<version>)

Gradle (C:\Gradle\gradle-<version>)

Click **Save**



Experiment 6: Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests.

Overview:

What is a CI Pipeline?

A **Continuous Integration (CI) Pipeline** automates the process of building, testing, and integrating code changes every time code is committed to the repository. This pipeline: Automatically checks out the latest code.

Compiles the application.

Runs tests to catch errors early.

Notifies the team of build/test results.

Why Use Jenkins for CI?

Automation: Jenkins automates the build and test cycle, reducing manual intervention.

Immediate Feedback: Developers get rapid notifications of any integration issues.

Extensibility: With hundreds of plugins available, Jenkins can integrate with version control systems, build tools (Maven, Gradle), testing frameworks, and more.

Pipeline as Code: Using Jenkins Pipelines (defined in a `Jenkinsfile`), you can manage the CI process as part of your source code repository.

Setting Up a CI Pipeline with Jenkins (Freestyle Project)

This section explains how to create a CI pipeline as a Freestyle project that integrates with a Maven or Gradle project

Step 1: Create a New Jenkins Job

1. Log into Jenkins:

- Open your web browser and navigate to your Jenkins URL (e.g.,

`http://localhost:8080` or your cloud instance URL).

Log in with your admin credentials.

2. Create a New Job:

On the Jenkins dashboard, click on “**New Item**”.

Enter an Item Name: For example, `Maven-CI` (or `Gradle-CI` if you prefer Gradle).

Select “Freestyle project”.

Click “**OK**”.

Step 2: Configure Source Code Management (SCM)

1. Select SCM:

In the job configuration page, scroll down to the “**Source Code Management**” section.

Select “**Git**” (if using Git for version control).

2. Enter Repository Details:

Repository URL: Enter the URL of your Git repository (for example,

`https://github.com/yourusername/your-maven-project.git`).

Credentials: If your repository is private, click “**Add**” to provide the necessary credentials.

Optionally, specify the **Branch Specifier** (e.g., `*/main`).

Step 3: Add Build Steps

A. For a Maven Project

1. Add Maven Build Step:

Scroll down to “**Build**” and click on “**Add build step**”.

Select “**Invoke top-level Maven targets**”.

Goals: In the Goals field, enter:

```
clean package
```

This command instructs Maven to clean the previous build artifacts, compile the code, run tests, and package the application into a JAR/WAR file.

Optionally, set the **POM File** location if it is not in the default location (`pom.xml`).

B. For a Gradle Project

1. Add Gradle Build Step:

If you are integrating a Gradle project, click on “**Add build step**” and choose “**Invoke Gradle script**” (this option might be available if you have installed a Gradle plugin in Jenkins).

Tasks: In the tasks field, enter:

```
clean build
```

This instructs Gradle to clean previous build outputs and then build the project, running tests along the way.

Switches: If needed, you can add additional flags (for example, `--info` or `--stacktrace` for more detailed output).

Objective

- To setup a continuous integration pipeline in Jenkins, integrate it with git, and run selenium java tests using maven.

Prerequisites

- Jenkins is installed and running on your system. If not, refer to [Experiment 5].
- Git is installed and configured in Jenkins. (Verify with `git --version`).
- Maven is installed and configured in Jenkins. (Check with `mvn --version`).

```

Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Saurav>git --version
git version 2.39.0.windows.2

C:\Users\Saurav>java --version
java 17.0.4 2022-07-19 LTS
Java(TM) SE Runtime Environment (build 17.0.4+11-LTS-179)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.4+11-LTS-179, mixed mode, sharing)

C:\Users\Saurav>mvn --version
Apache Maven 3.8.6 (84538c9988a25aec085021c365c560670ad80f63)
Maven home: D:\Softwares\apache-maven-3.8.6
Java version: 17.0.4, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-17.0.4
Default locale: en_IN, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

C:\Users\Saurav>gradle -v

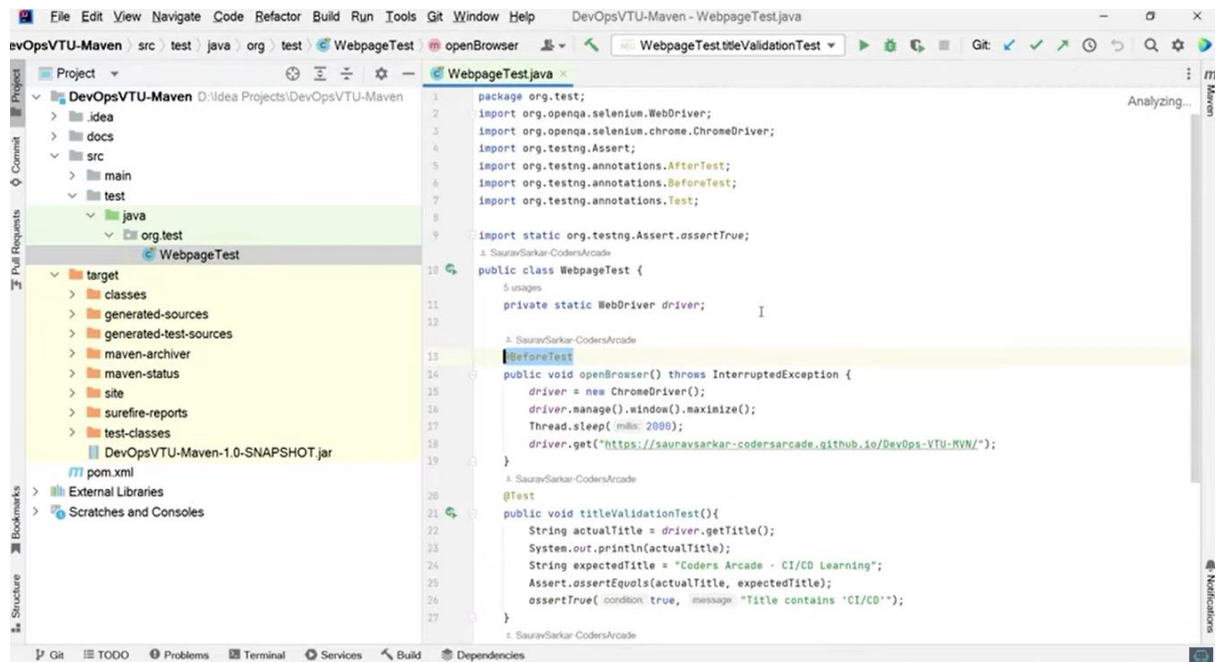
```

The screenshot shows a GitHub repository page for 'DevOps-VTU-MVN' by SauravSarkar-CodersArcade. The repository is public and has 1 branch and 0 tags. The commit history shows several commits, including adding files for the mvn life cycles site & deploy. The repository structure is as follows:

File/Folder	Description	Last Commit
.idea	Added the docs folder for deployment	last week
docs	Added the docs folder for deployment	last week
src	Added all files till mvn life cycles site & deploy. All finalized	last week
target	Added all files till mvn life cycles site & deploy. All finalized	last week
pom.xml	Added all files till mvn life cycles site & deploy. All finalized	last week

The README section is currently empty, with a button to 'Add a README'.

- Selenium Maven Project is ready with test cases (src/test/java).



- Project is stored in two places:
- Locally on your system (e.g., D:\Idea Projects\MVNGRDLEMO).
- Pushed to GitHub with a valid repository link.
- Jenkins has access to the GitHub repository (via credentials)
- Open terminal

git status

git remote show origin

```
nothing to commit, working tree clean
PS D:\Idea Projects\DevOpsVTU-Maven> git remote show origin
* remote origin
Fetch URL: https://github.com/SauravSarkar-CodersArcade/DevOps-VTU-MVN.git
Push URL: https://github.com/SauravSarkar-CodersArcade/DevOps-VTU-MVN.git
HEAD branch: master
Remote branch:
  master tracked
Local branch configured for 'git pull':
  master merges with remote master
Local ref configured for 'git push':
  master pushes to master (up to date)
PS D:\Idea Projects\DevOpsVTU-Maven>
```

mvn test

- Copy the url of your project
- Open cmd → path of jenkins.war

Running a Selenium java test from local Maven project.

Step 1: Create a New Jenkins Job

Open jenkins

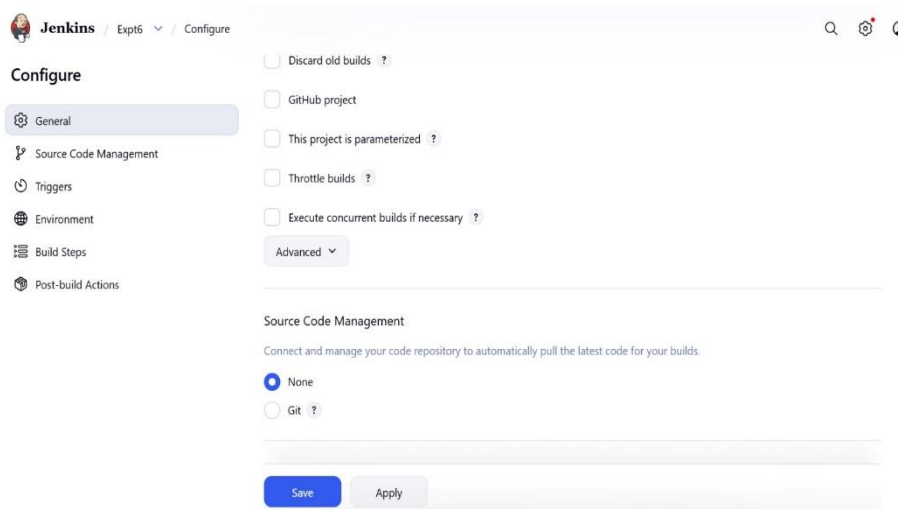
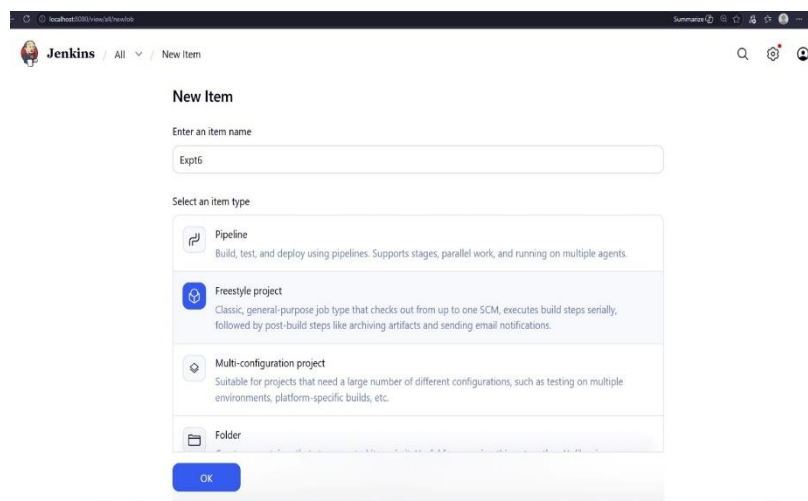
Go to jenkins dashboard

Click new item

Enter project name

Select free style project

Click ok.



Step 2: Configure the build step

1. scroll to build
2. Click add build step
3. Click on execute windows batch command
4. Enter the project directory copied from terminal
C:
cd C:\Users\raushan\IdeaProjects\DevOps
mvn test
→ Apply → save.
5. Click on build now to execute the test

The image shows two parts of a Jenkins setup. The top part is the Jenkins 'Configuration' page for a job named 'DevOps-VTU-Exp-6-Selenium-Maven-Jenkins-local-Run'. The 'Environment' tab is selected in the left sidebar. Under 'Build Steps', the 'Execute Windows batch command' step is configured with the following command:

```
C:
cd C:\Users\vaish\IdeaProjects\DevOps
mvn test
```

The bottom part of the image is a terminal window showing the output of the Jenkins build. The build is successful, and the output includes the following log entries:

```
[INFO] --- surefire:3.2.5:test (default-test) @ MUNGRDCDEMO ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit3Unit3Provider
[INFO]
[INFO] T E S T S
[INFO] -----
[INFO] Running org.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in org.example.AppTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.749 s
[INFO] Finished at: 2025-04-10T15:38:09+05:30
[INFO]
[INFO] Finished: SUCCESS
```

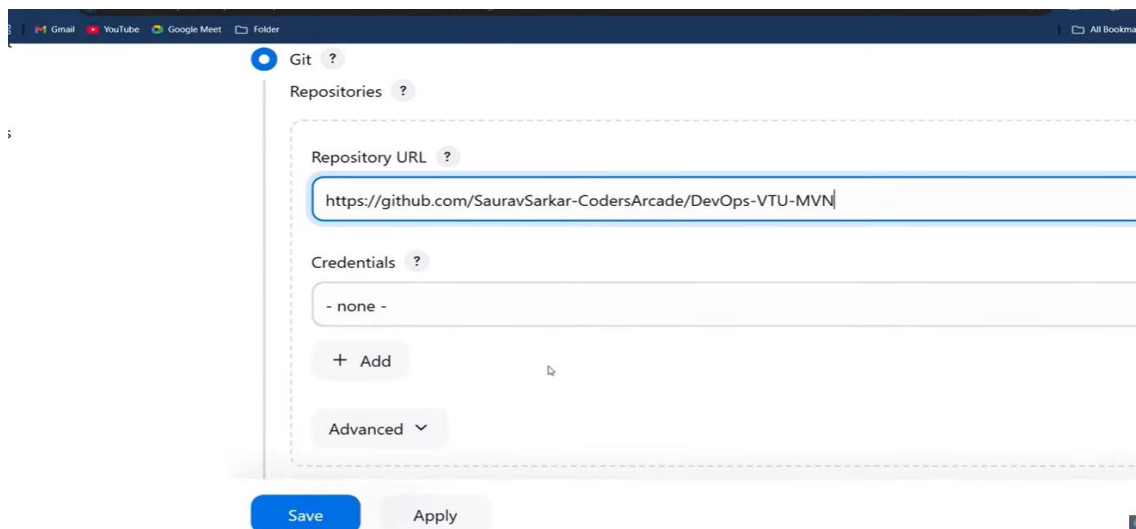
Connecting Github with Jenkins

Step 1: Set Up a New Jenkins Job for GitHub Project

1. Go to Jenkins Dashboard → Click New Item.
2. Enter a project name → Select Freestyle Project.
3. Click OK.

Step 2: Configure Git Repository in Jenkins

1. Under Source Code Management, select Git.
2. Enter your GitHub repository URL (e.g., <https://github.com/SauravSarkar-CodersArcade/DevOps-VTU-MVN>).
 - How to add github credentials to Jenkins



Follow these steps 3:

- Click on add credentials → select global credentials.
- Select kind as → username with pw → scope keep as it is.
- Enter your github or email username and pw
- Id → Github login ID (optional)

Add description → Apply and save

3. Select the Git credentials configured earlier.
4. Select branch master → Apply → Save.
5. Click on Build to execute the test.

- (This is how you can connect github with jenkins.)

Step 4: Trigger the Build

1. Click Build Now to fetch the code from GitHub and execute the Selenium tests.

Check the Console Output to verify test execution

The screenshot shows the Jenkins web interface for a build named 'exp6demo'. The left sidebar contains navigation links: Status, </> Changes, Console Output (which is selected), Edit Build Information, Delete build '#1', Timings, and Git Build Data. The main area is titled 'Console' and displays the build's log output. At the top right of the console area are buttons for 'Download', 'Copy', and 'View as plain text'. The log output details the build process, including cloning a repository from GitHub, fetching upstream changes, and checking out a specific revision. The build concludes with the message 'Finished: SUCCESS'.

```
Started by user SamhithaV
Running as SYSTEM
Building in workspace C:\Users\BIET\.jenkins\workspace\exp6demo
The recommended git tool is: NONE
using credential 9ebd5162-3fda-41fd-aac4-ff48ca3a9da6
Cloning the remote Git repository
Cloning repository https://github.com/SamhithaV-18/Exp2demo
> git.exe init C:\Users\BIET\.jenkins\workspace\exp6demo # timeout=10
Fetching upstream changes from https://github.com/SamhithaV-18/Exp2demo
> git.exe --version # timeout=10
> git --version # 'git version 2.53.0.windows.2'
using GIT_ASKPASS to set credentials
> git.exe fetch --tags --force --progress -- https://github.com/SamhithaV-18/Exp2demo +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe config remote.origin.url https://github.com/SamhithaV-18/Exp2demo # timeout=10
> git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git.exe rev-parse "refs/remotes/origin/master^{commit}" # timeout=10
Checking out Revision 0fd5c813afb3f8c29698193174cb4e13050f011 (refs/remotes/origin/master)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 0fd5c813afb3f8c29698193174cb4e13050f011 # timeout=10
Commit message: "deploy"
First time build. Skipping changelog.
Finished: SUCCESS
```

Experiment 7: Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook

What Is Ansible?

Ansible is an open-source IT automation and configuration management tool. It allows you to manage multiple servers and perform tasks such as:

Configuration Management: Automate the configuration of servers.

Application Deployment: Deploy applications consistently.

Orchestration: Coordinate complex IT workflows and processes

Key Concepts in Ansible

Inventory:

An inventory is a file (usually in INI or YAML format) that lists the hosts (or groups of hosts) you want to manage. It tells Ansible which machines to target.

Playbook:

A playbook is a YAML file that defines a set of tasks to be executed on your target hosts. It is the heart of Ansible automation. In a playbook, you specify:

Hosts: The target machines (or groups) on which the tasks should run.

Tasks: A list of actions (using modules) that should be executed.

Modules: Reusable, standalone scripts that perform specific actions (e.g., installing packages, copying files, configuring services).

Modules:

Ansible comes with a large collection of built-in modules (such as `apt`, `yum`, `copy`, `service`, etc.). These modules perform specific tasks on target hosts. You can also write custom modules if needed.

Why Use Ansible?

Agentless: Ansible uses SSH to communicate with target hosts, so no agent needs to be installed on them.

Simplicity: Playbooks use simple YAML syntax, making them easy to write and understand.

Idempotence: Ansible tasks are idempotent, meaning running the same playbook multiple times yields the same result, ensuring consistency.

Scalability: Ansible can manage a small number of servers to large infrastructures with hundreds or thousands of nodes.

2. Installing Ansible on Ubuntu

Before writing a playbook, you need to install Ansible on your control machine (your local Ubuntu system).

Step 1: Update Your System

Open your terminal and run:

```
sudo apt update sudo apt upgrade -y
```

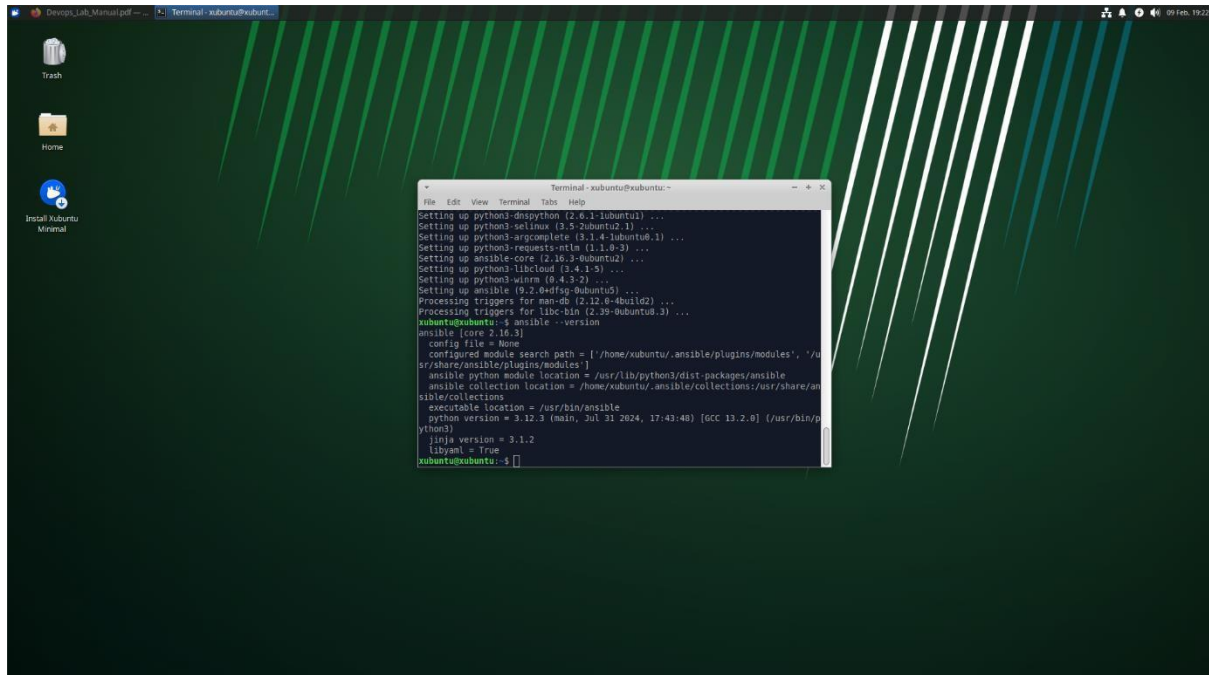
Step 2: Install Ansible

Install Ansible using `apt`:

```
sudo apt install ansible -y
```

Verify the installation by checking the version:

```
ansible --version
```



```
Setting up python3-dnspython (2.6.1-1ubuntu1) ...
Setting up python3-selinux (3.5.2ubuntu2.1) ...
Setting up python3-argcomplete (3.1.4-1ubuntu0.1) ...
Setting up python3-requests-netto (1.1.0-3) ...
Setting up ansible-core (2.16.3-0ubuntu2) ...
Setting up python3-libcloud (3.4.1-5) ...
Setting up python3-wsman (0.4.3-2) ...
Setting up ansible (9.2.0+dfsg-0ubuntu5) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu3) ...
xubuntu@xubuntu:~$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/xubuntu/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/xubuntu/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Jul 31 2024, 17:43:48) [gcc 13.2.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
xubuntu@xubuntu:~$
```

3.Creating an Ansible Inventory

An inventory file lists the hosts you want to manage. For this experiment, you can use the local host.

3.1. Create an Inventory File

Open your text editor to create a file called hosts.ini:

```
gedit hosts.ini
```

3.2.Add the following content to define the local host:

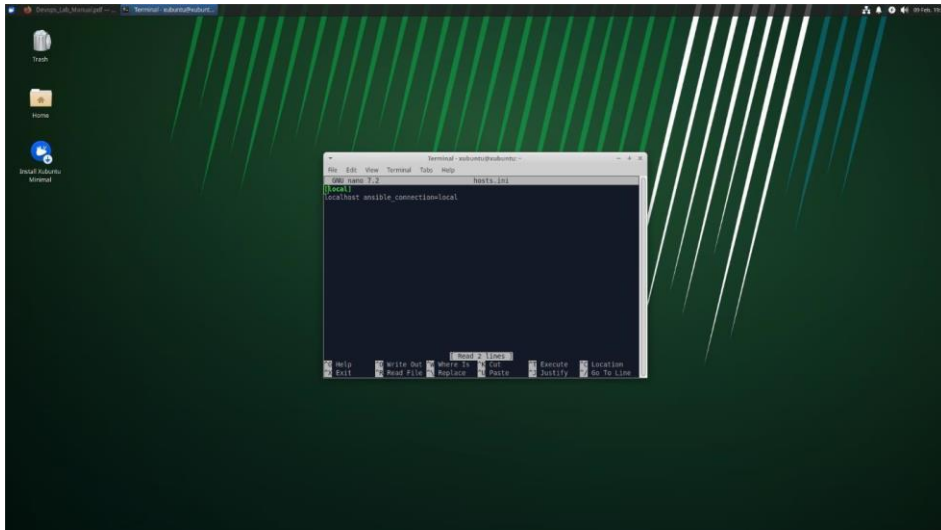
```
[local]
```

```
localhost ansible_connection=local
```

Explanation:

- [local] is a group name.
- localhost is the target host.
- ansible_connection=local tells Ansible to execute commands on the local machine without

3.3. Save and exit the file.



4. Automated Server Configurations

While our experiment covered the basics, here's how you can extend it:

Configuring Services:

Use modules like `service` to start, stop, or restart services. For example, you can automate the configuration of web servers (e.g., Apache or Nginx).

Managing Files and Templates: Use the `copy` or `template` modules to deploy configuration files across your servers. This is useful for maintaining consistent configuration settings.

User and Group Management: The `user` and `group` modules allow you to create or modify user accounts, ensuring that the correct permissions and roles are applied automatically.

Advanced Orchestration:

Ansible playbooks can include conditionals, loops, and error handling to manage more complex setups, ensuring idempotence and consistency across your infrastructure.

Example: Automating a Web Server Configuration

If you wanted to automate the configuration of an Nginx server, your playbook might include tasks such as:

```
- name: Configure Nginx Web Server
hosts: webserver
become: yes
tasks:
- name: Update apt cache
apt:

update_cache: yes
- name: Install Nginx
apt:

name: nginx
state: present
- name: Copy Nginx configuration file
template:

src: nginx.conf.j2
dest: /etc/nginx/nginx.conf
```

```
notify:
- Restart Nginx
handlers:
- name: Restart Nginx
service:
```

```
name: nginx
```

```
state: restarted
```

5. Writing a Basic Ansible Playbook

You will now create a simple playbook that performs two common tasks:

- a) Updating the apt cache
- b) Installing a package

Step1. Create the Playbook File

Open your text editor to create a file called setup.yml:

```
gedit setup.yml
```

Add the following YAML content:

```
---
```

```
- name: Basic Server Setup
```

```
hosts: localhost
```

```
become: yes
```

```
tasks:
```

```
- name: Update apt cache
```

```
apt:
```

```
update_cache: yes
```

```
- name: Install Apache
```

```
apt:
```

```
name: apache2
```

```
state: present
```

```
- name: Start Apache service
```

```
service:
```

```
name: apache2
```

```
state: started
```

```
enabled: yes
```

```
- name: Create web page
```

copy:

dest: /var/www/html/index.html

content: "<h1>Ansible Web Server Working</h1>"

Explanation:

name: Provides a descriptive name for the play.

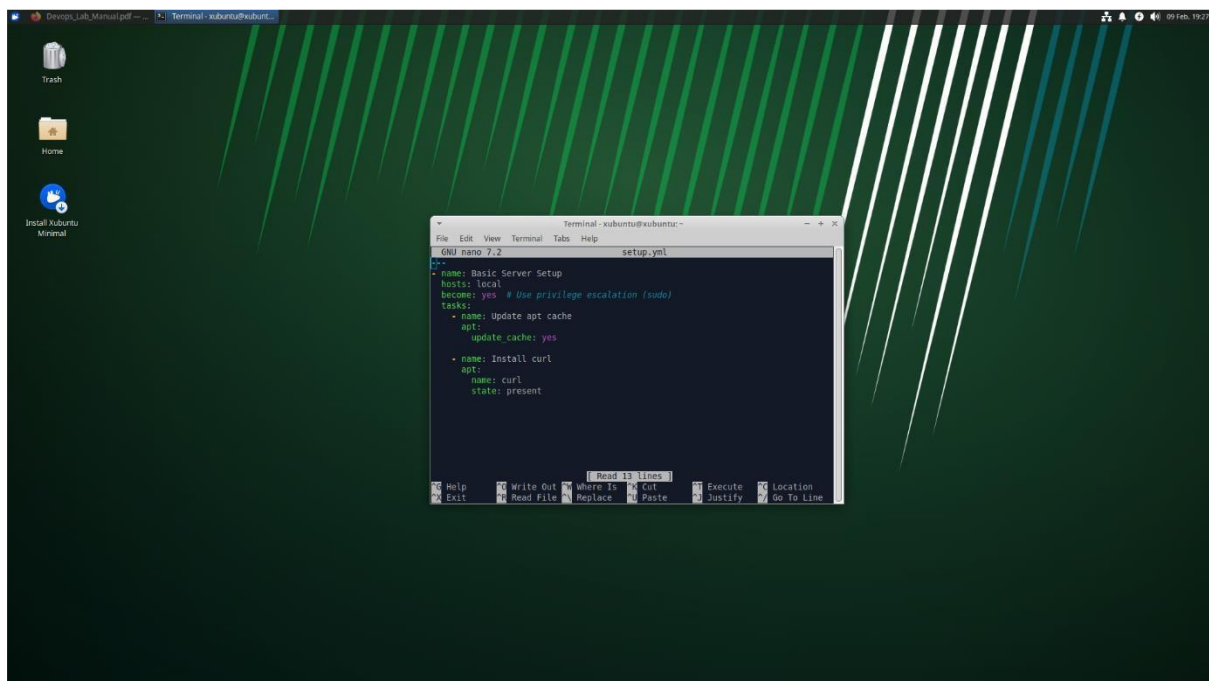
hosts: Specifies the group or hosts from the inventory file.

become: yes: Uses sudo to perform tasks that require elevated privileges.

Tasks Section:

- Update apt(advanced package tool) cache: Uses the apt module to update the package cache.
- Install curl: Uses the apt module to install the curl package if it isn't already installed.

3. Save and exit the file



Running the Ansible Playbook

6.1. Execute the Playbook

In your terminal, run the following command:

```
sudo ansible-playbook -i hosts.ini setup.yml
```

Explanation:

ansible-playbook: The command to run an Ansible playbook.

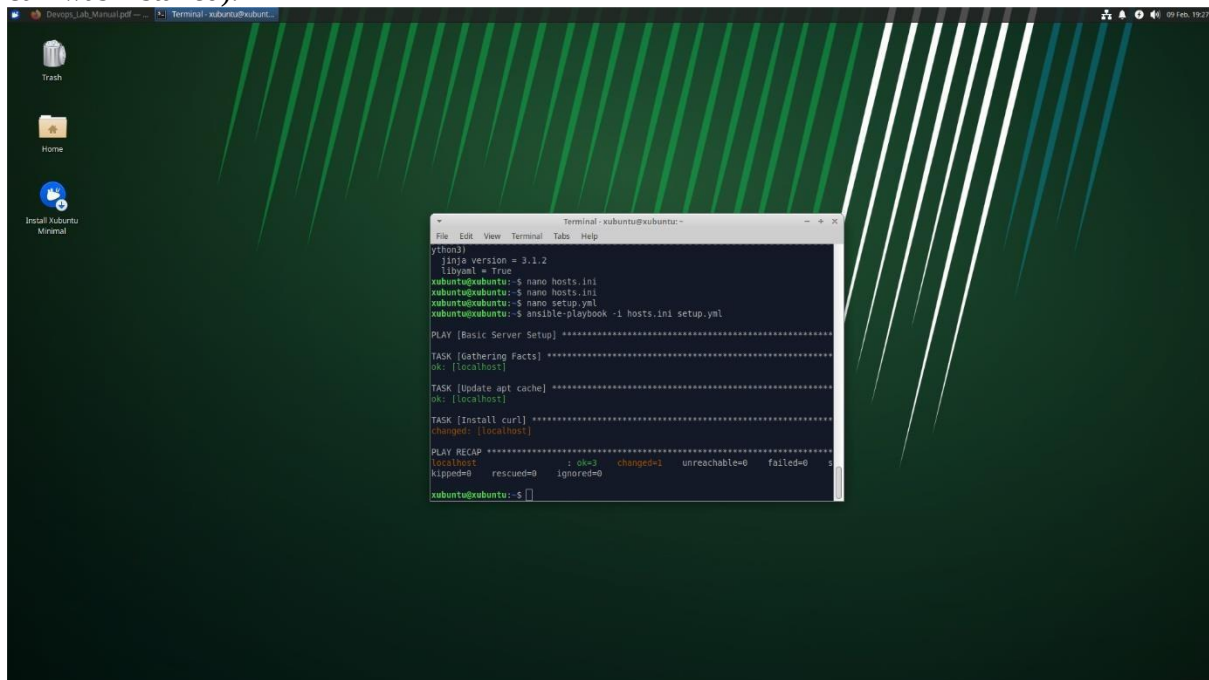
-i hosts.ini: Specifies the inventory file.

setup.yml: The playbook file you just created.

Expected Output:

□ An output that details each task:

For example, a task summary might show “ok=2 changed=1” (if the apt cache was updated and curl was installed).

A screenshot of a Linux desktop environment with a dark green background and diagonal white lines. On the left, there are icons for 'Trash', 'Home', and 'Install Ubuntu Minimal'. In the center, a terminal window titled 'Terminal - xubuntu@xubuntu:~' is open. The terminal shows the following commands and output:

```
python3
jshjs version = 3.1.2
libyaml = True
xubuntu@xubuntu:~$ nano hosts.ini
xubuntu@xubuntu:~$ nano hosts.ini
xubuntu@xubuntu:~$ nano setup.yml
xubuntu@xubuntu:~$ ansible-playbook -i hosts.ini setup.yml

PLAY [Basic Server Setup] *****
TASK [Gathering Facts] *****
ok: [localhost]

TASK [Update apt cache] *****
ok: [localhost]

TASK [Install curl] *****
changed: [localhost]

PLAY RECAP *****
localhost : ok=3  changed=1  unreachable=0  failed=0  s
kipped=0  rescued=0  ignored=0
xubuntu@xubuntu:~$
```

Experiment 8: Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins

1. Overview

In this experiment, you will:

Set up a Jenkins job to automatically build a Maven project from source control.

Archive the build artifact (a JAR file) produced by Maven.

Integrate an Ansible deployment step within Jenkins (using a post-build action) to deploy the artifact to a target location.

Verify that the artifact is deployed successfully.

This exercise demonstrates how Continuous Integration (CI) and automated configuration management can work together to streamline the build-and-deploy process.

2. Prerequisites

Before you begin, ensure that:

Jenkins is installed, running, and accessible (locally or on the cloud).

Maven Project: You have a Maven project available in a Git repository (or stored locally). For this example, we will assume you are using the “HelloMaven” project generated in Experiment 2/4.

Git Repository: The Maven project is committed to a Git repository (e.g., on GitHub) so Jenkins can pull the latest code.

Ansible Installed: Ansible is installed on your control machine (or the Jenkins server) and you have created a basic inventory file (e.g., `hosts.ini`).

Tip: Verify installations and repository access before starting this exercise.

Create a New Jenkins Job (Freestyle Project)

```
cd path/to/HelloMaven
```

```
git init
```

```
git add .
```

```
git commit -m "Initial commit of HelloMaven project"
```

Then push it to your repository (GitHub, GitLab, etc.).

Verify the Project Structure:

Your project should have a standard Maven layout:

```
HelloMaven/
```

```
├─ pom.xml
```

```
└─ src
```

```
├─ main/java/com/example/App.java
```

```
└─ test/java/com/example/AppTest.java
```

Step 1: Preparing the Maven Project

1. Ensure Your Maven Project Is in Version Control: If your "HelloMaven" project is not already in a Git repository, navigate to its root and initialize Git:

Log into Jenkins: Open your browser and navigate to your Jenkins URL (e.g., <http://localhost:8080>).

Create a New Job:

- Click "New Item" on the Jenkins dashboard.
- Enter Item Name: e.g., HelloMaven-CI Select "Freestyle project" and click "OK".

2. Configure Source Code Management (SCM)

1. Scroll to the "Source Code Management" Section:
2. Select "Git".
3. Repository URL: Enter your repository URL: <https://github.com/devops-ds/your-maven-project.git>
4. Credentials: If the repository is private, click "Add" and provide the necessary credentials. (Not needed for the given public repository).
5. Branch Specifier: (e.g., */main or */master).

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

Credentials ?

+ Add

Advanced ▾

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

+ Add Branch

Repository browser ?

3. Add Build Steps

1. Scroll Down to the "Build" Section.
2. Add the Maven Build Step: Click "Add build step" and select "**Window Batch Command**".

3. Configure the Windows Batch Command:

mvn clean package

1. Add the Maven Build Step: Click "Add build step" and select "**Windows Batch Command**".

2. Configure the Windows Batch Command:

mkdir C:\Users\raushan\.jenkins\deployment1

copy target*.jar C:\Users\raushan\.jenkins\deployment1\

The screenshot shows two instances of the 'Execute Windows batch command' build step configuration in Jenkins. Each instance has a title bar with three dots and a question mark icon. Below the title bar is the label 'Command' and a link 'See the list of available environment variables'. A text input field contains the command 'mvn clean package'. Below the input field is a button labeled 'Advanced' with a downward arrow. The second instance of the configuration is identical but contains the commands 'mkdir C:\Users\raushan\.jenkins\deployment1' and 'copy target*.jar C:\Users\raushan\.jenkins\deployment1\'. The 'mkdir' command has a red squiggly underline under the path.

4. Archiving the Artifact

After the Maven build completes, you need to archive the generated artifact so that it can be used later by the deployment process.

1. Scroll Down to the "Post-build Actions" Section:

- Click "Add post-build action" and select "Archive the artifacts".

2. Configure Artifact Archiving:

- Files to Archive: Type: target/*.jar
- This pattern tells Jenkins to archive any JAR file found in the target directory.

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Archive the artifacts ?

Files to archive ?

target/*.jar

Advanced ▾

+ Add post-build action

5. Save the Jenkins Job:

Click on Apply and Save.

6. Writing an Ansible Playbook for Deployment

Create an Ansible playbook that deploys the Maven artifact (the JAR file) generated by Jenkins to a target directory.

A. Create an Inventory File

- If you haven't already, create an inventory file (e.g., hosts.ini) that targets the deployment machine. For a local deployment, use:

```
gedit hosts.ini
```

```
[local]
```

```
localhost ansible_connection=local
```

B. Create the Deployment Playbook

1. Open Your Text Editor and create a file called deploy.yml:

```
gedit deploy.yml
```

2. Enter the Following YAML Content:

```
---
```

```
- name: Deploy Maven Artifact (Jenkins User Accessible)
```

```
  hosts: local
```

```
  tasks:
```

```
    - name: Ensure deployment directory exists
```

```
      ansible.builtin.file:
```

```
        path: "C:\\Users\\raushan\\.jenkins\\deployment"
```

```
        state: directory
```

```
    - name: Copy the artifact to the deployment directory
```

```
      ansible.builtin.copy:
```

```
        src: "C:\\Users\\raushan\\.jenkins\\workspace\\HelloMaven-CI\\target\\DAY2-EXP-2-1.0-SNAPSHOT.jar"
```

```
        dest: "C:\\Users\\raushan\\.jenkins\\deployment\\HelloMaven.jar"
```

remote_src: yes

3. Save and Exit the File

7. Testing the Complete Pipeline

1. Trigger a Build in Jenkins:

Navigate to your Jenkins job (HelloMaven-CI) and click "Build Now".

Monitor the build history. Once the build completes, click the build number (e.g., #1) and check the Console Output.

Look for messages indicating that:

- The Maven build ran successfully.
- The shell command executed the Ansible playbook.
- The artifact was archived.

2. Verify Deployment:

Open file explorer and verify that the artifact has been copied to the destination directory. Check in terminal, run the below command in terminal:

```
ls -l /mnt/c/Users/raushan/.jenkins/deployment1
```

The output should list the deployed JAR file.

```
raushan@LAPTOP-BSREFJ26:~$ ls -l /mnt/c/Users/raushan/.jenkins/deployment1
total 12
-rwxrwxrwx 1 raushan raushan 10174 Mar 11 20:13 DAY2-EXP-2-1.0-SNAPSHOT.jar
raushan@LAPTOP-BSREFJ26:~$ |
```

Experiment 9: Introduction to Azure DevOps: Overview of Azure DevOps Services, Setting Up an Azure DevOps Account and Project

Overview of Azure DevOps

Azure DevOps is a comprehensive suite of cloud-based services designed to support the entire software development lifecycle. It provides tools for planning, developing, testing, delivering, and monitoring applications. Here are the primary services offered:

- **Azure** **Repos:**
A set of version control tools that allow you to host Git repositories or use Team Foundation Version Control (TFVC). It offers collaboration features such as pull requests, branch policies, and code reviews.
- **Azure** **Pipelines:**
A CI/CD service that helps automate builds, tests, and deployments. It supports multiple languages, platforms, and can run on Linux, Windows, or macOS agents.
- **Azure** **Boards:**
A work tracking system that helps teams manage work items, sprints, backlogs, and Kanban boards. It facilitates agile planning and reporting.
- **Azure** **Test** **Plans:**
Provides a solution for managing and executing tests, capturing data about defects, and tracking quality.
- **Azure** **Artifacts:**
Allows you to create, host, and share packages (such as Maven, npm, NuGet, and Python packages) with your team, integrating package management into your CI/CD pipelines.

These services integrate with each other and with popular third-party tools to create a cohesive DevOps ecosystem.

Setting Up an Azure DevOps Account

Before you can start using Azure DevOps services, you need to set up an account and create an organization. Follow these steps:

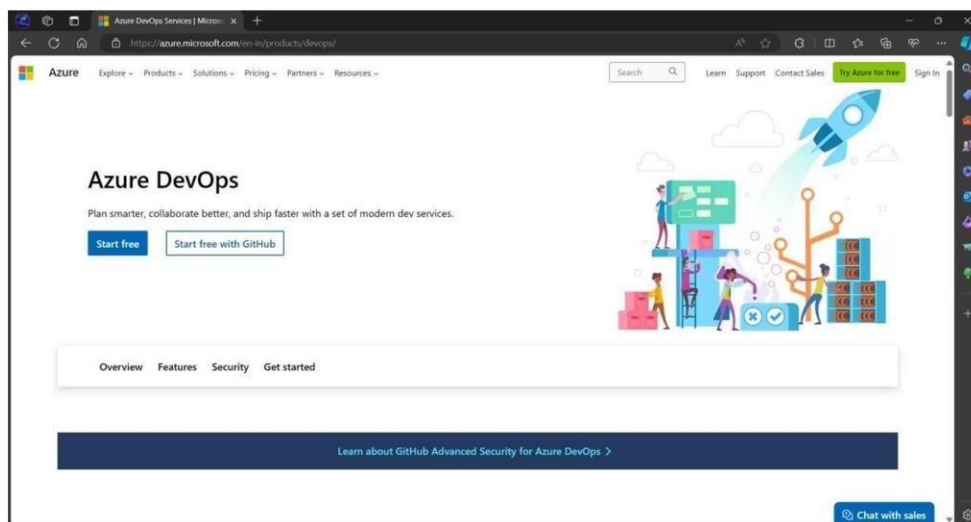
Step 1: Sign Up for an Azure DevOps Account

1. Open Your Web Browser:

- Navigate to the Azure DevOps website: <https://dev.azure.com>.

2. Sign In or Create a Microsoft Account:

- If you already have a Microsoft account (such as Outlook, Hotmail, or Office 365), click “Sign in”.
- If you do not have a Microsoft account, click “Create one!” and follow the instructions to create a new Microsoft account.



3. Accept the Terms and Conditions:

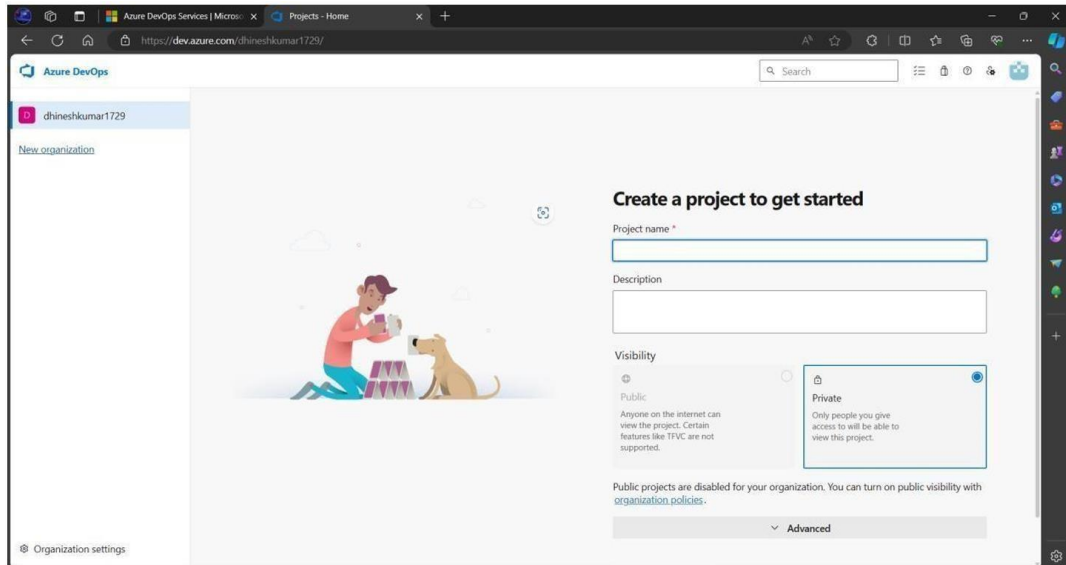
- Review and accept the terms if prompted.

Step 2: Create an Azure DevOps Organization

1. Create a New Organization:

- Once signed in, you will be prompted to create an Azure DevOps organization.
- Enter a unique name for your organization (e.g., YourCompanyDevOps or MyPersonalOrg).

- **Select a Region:** Choose the geographic region where your data will be stored (select the one closest to you for optimal performance).
- Click **“Continue”** or **“Create”**.



2. Review Your Organization’s Dashboard:

- Once created, you will see an overview dashboard for your organization. This dashboard provides navigation links to Repos, Pipelines, Boards, Test Plans, and Artifacts.

3. Creating an Azure DevOps Project

After setting up your organization, the next step is to create a project. A project in Azure DevOps is a container for all your source code, pipelines, boards, and other resources.

Step 1: Create a New Project

1. Navigate to “New Project”:

- On your organization’s dashboard, click the **“New Project”** button.

2. Configure Your Project:

- **Project Name:** Enter a descriptive name for your project (e.g., HelloDevOps).
- **Description:** Optionally, provide a brief description (e.g., “A sample project to demonstrate Azure DevOps services”).
- **Visibility:**

- Choose “**Private**” if you want to restrict access to your project.
- Choose “**Public**” if you are okay with the project being accessible to anyone.
- **Advanced Options (Optional):** You can choose a version control system (Git is the default) and a work item process (Agile, Scrum, or Basic). For most beginners, the defaults are recommended.
- Click “**Create**”.

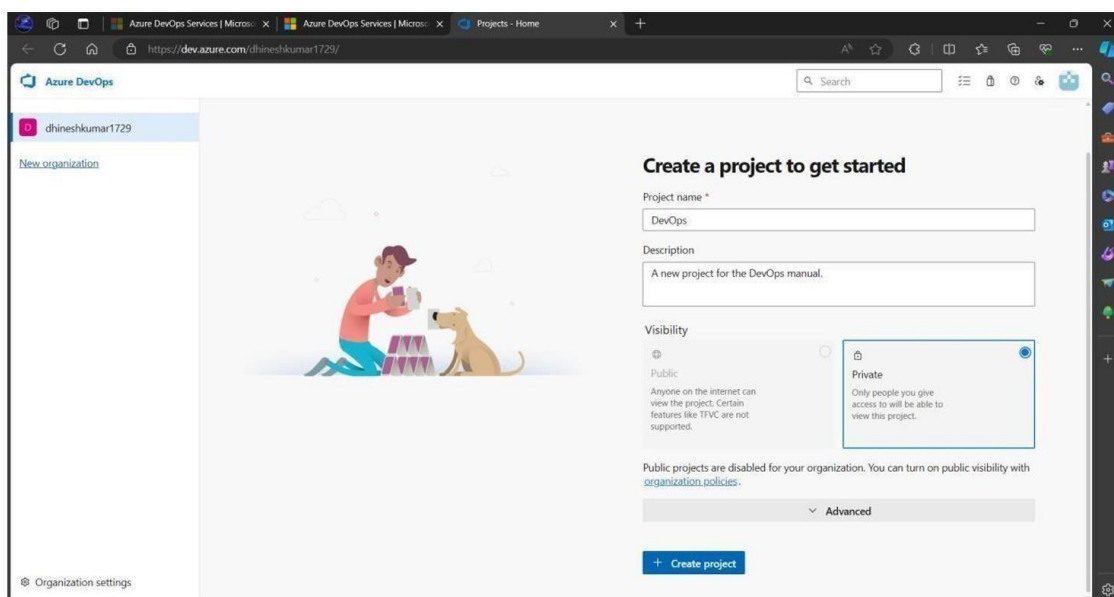
Step 2: Explore Your Project Dashboard

1. Project Overview:

- Once your project is created, you will be directed to the project dashboard. Here you will see navigation options for:
 - **Repos:** Where your code is stored.
 - **Pipelines:** For build and release automation.
 - **Boards:** For work tracking and agile planning.
 - **Test Plans:** For managing and running tests.
 - **Artifacts:** For hosting packages.

2. Familiarize Yourself with the Interface:

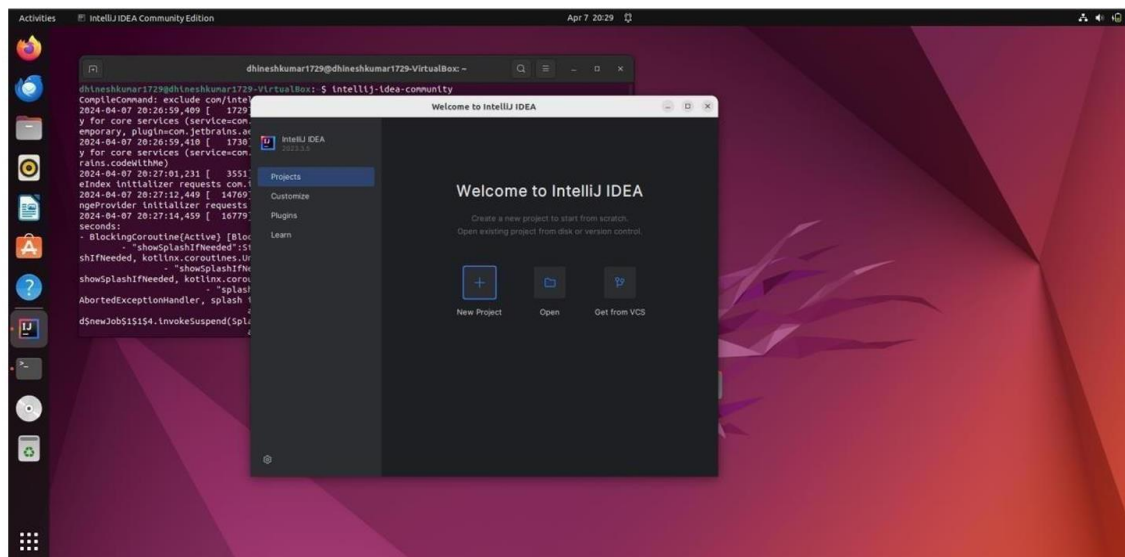
- Click through each section (e.g., Repos, Pipelines, Boards) to get a sense of the available features.



Experiment 10: Creating Build Pipelines: Building a Maven/Gradle Project with Azure Pipelines, Integrating Code Repositories (e.g., GitHub, Azure Repos), Running Unit Tests and Generating Reports

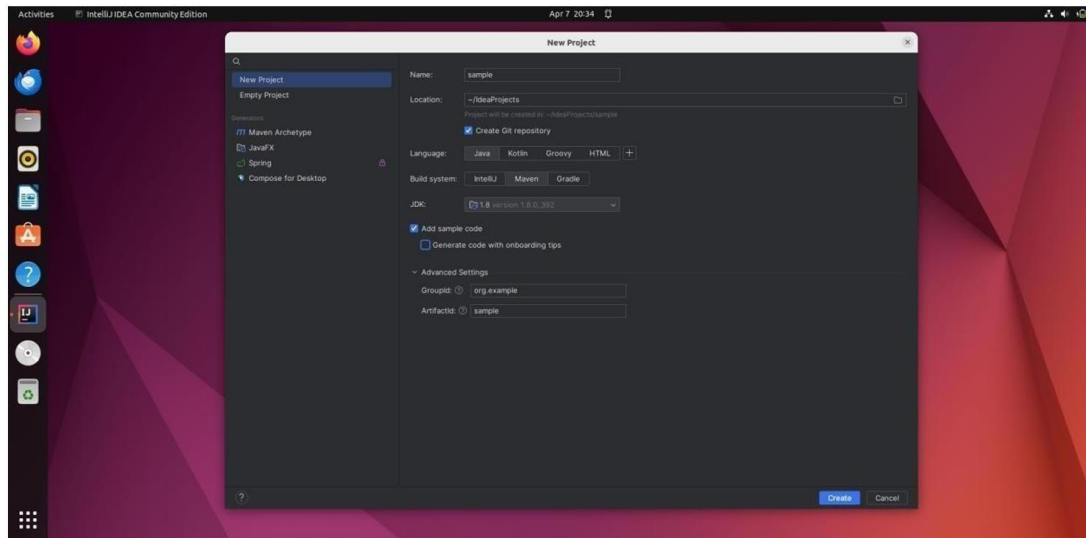
Step 1: Installing IntelliJ IDEA

1. Download IntelliJ Idea from <https://www.jetbrains.com/idea/download/?section=linux> and activate a free trial for 30 days or through the following command. `$ sudo snap install intellij-idea-community --community`
2. Open the IDE using the below command in the terminal. `$ intellij-idea-community`
3. Java is a pre-requisite for using this IDE



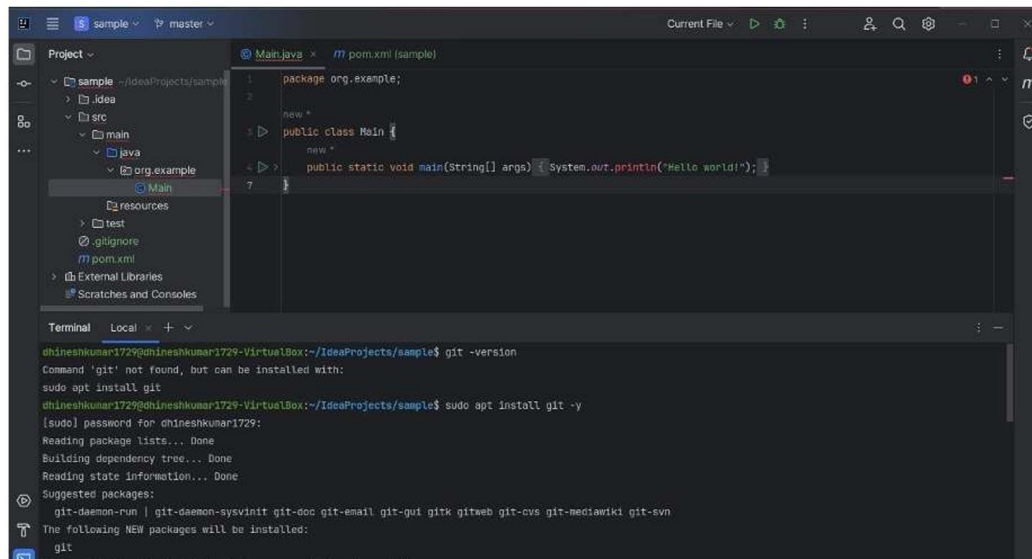
Step 2: Create a new Maven project using the IDE.

Create a project by clicking on the New Project option, choose an appropriate name and select the following features as mentioned in the image (Build system: Maven and Language: Java)



Step 3: Install GIT and configure it

1. Check whether git is installed using the following command. `$ git -version`
2. Install git using `$ sudo apt install git`



3. Once git has been installed execute the following commands one by one.

`$ git init $`

`git add .`

`$ git commit -m "First Commit"`

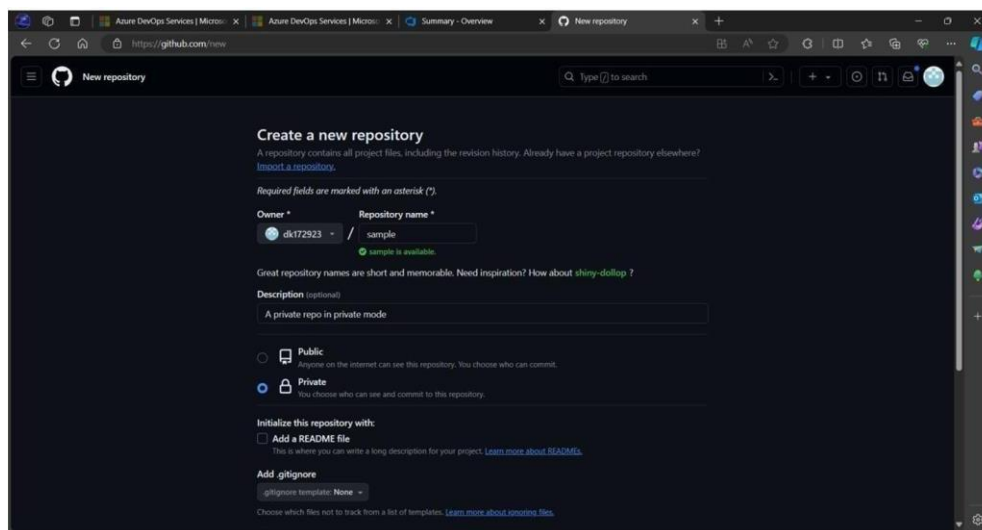
`$ git branch -M main`

Configure git using the below commands

```
$ git config --global user.name "Your_Name"
```

```
$ git config --global user.email "Your_Email_ID"
```

Go to <https://github.com> sign in with your account and create a new private repository with the same name as your maven project.



1. Copy the SSH URL for the created repository.

2. Now go back to IDE's terminal and execute the following commands

```
$ ssh-keygen -t rsa -b 4096 -C youremail@gmail.com
```

(press ENTER for all questions)

```
$ cat ~/.ssh/id_rsa.pub (copy the printed SSH key)
```

1. Now go to the SSH and GPG section in the GitHub settings option.

2. Click on the New SSH key button.

3. Choose a suitable name and paste the SSH key into the provided space.

4. Now get back to the IDE's terminal and type the following command

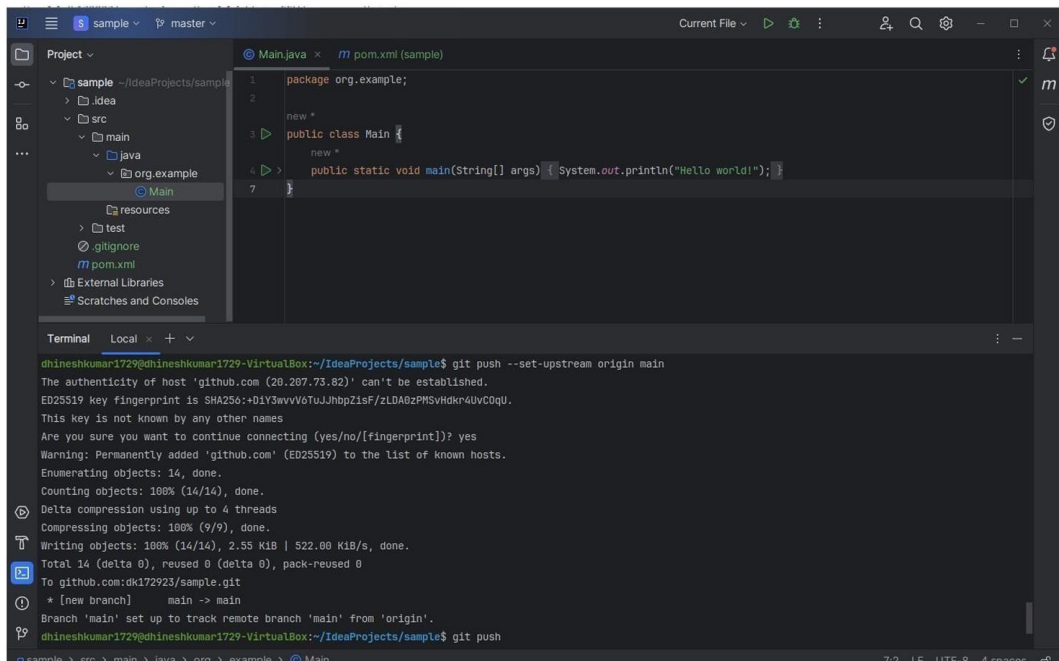
```
$ git remote set-url origin git@github.com:repo_owner/repo_name.git
```

(Here, git@github.com:dk172923/sample.git is the remote URL to use SSH
dk172923 – GitHub ID sample.git – repository name)

Now finally run the push commands to push the code to the remote repository from the local repository.

```
$ git push --set-upstream origin main (Type YES for prompted question)
```

\$ git push



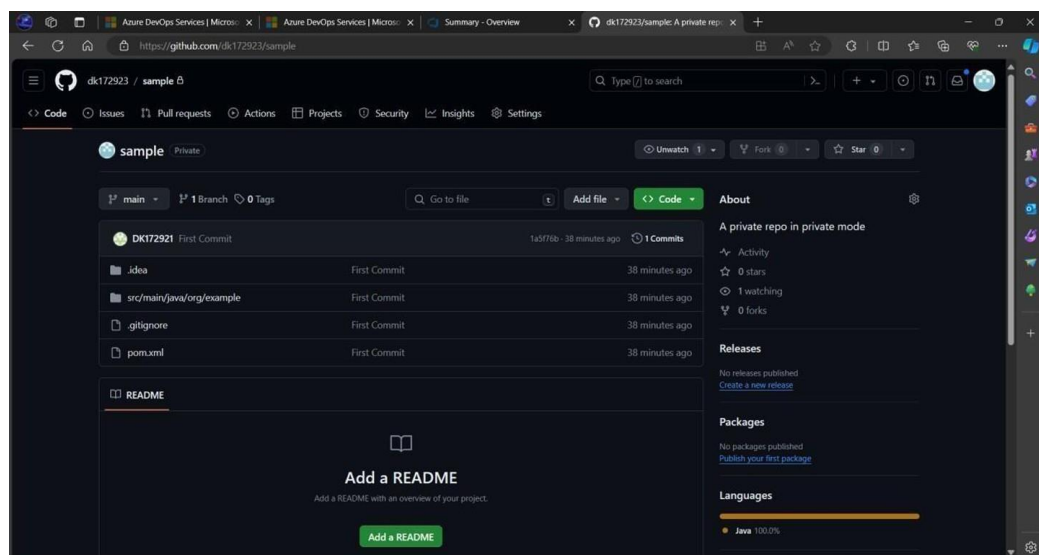
The screenshot shows an IDE with a project named 'sample'. The file explorer on the left shows the project structure: 'sample' -> 'src' -> 'main' -> 'java' -> 'org.example' -> 'Main'. The main editor shows 'Main.java' with the following code:

```
1 package org.example;
2
3 new *
4 public class Main {
5     new *
6     public static void main(String[] args) { System.out.println("Hello world!"); }
7 }
```

The terminal window at the bottom shows the output of the command 'git push --set-upstream origin main':

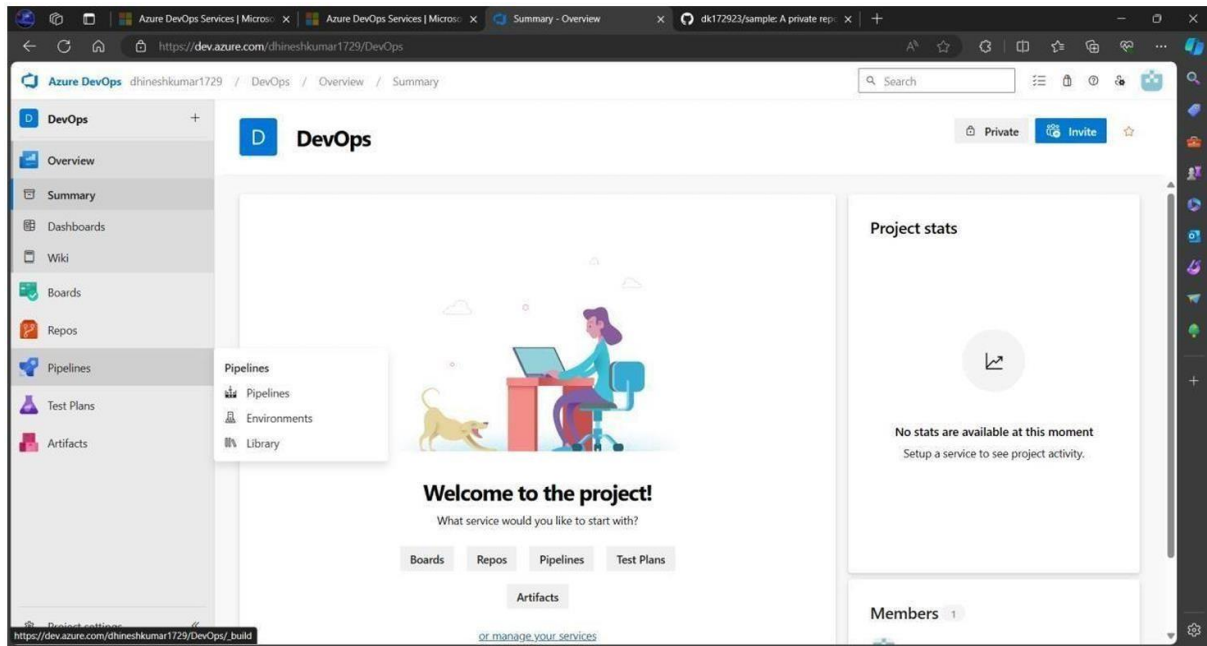
```
dhineshkumar1729@dhineshkumar1729-VirtualBox:~/IdeaProjects/sample$ git push --set-upstream origin main
The authenticity of host 'github.com (20.207.73.82)' can't be established.
ED25519 key fingerprint is SHA256:+D1Y3avvV6TuJhnpZlsF/zLDA8zPMSvHdkr4UvCOQu.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 4 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (14/14), 2.55 KiB | 522.00 KiB/s, done.
Total 14 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:dk172923/sample.git
 * [new branch]    main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
dhineshkumar1729@dhineshkumar1729-VirtualBox:~/IdeaProjects/sample$ git push
```

The code has been successfully pushed to the remote repository.

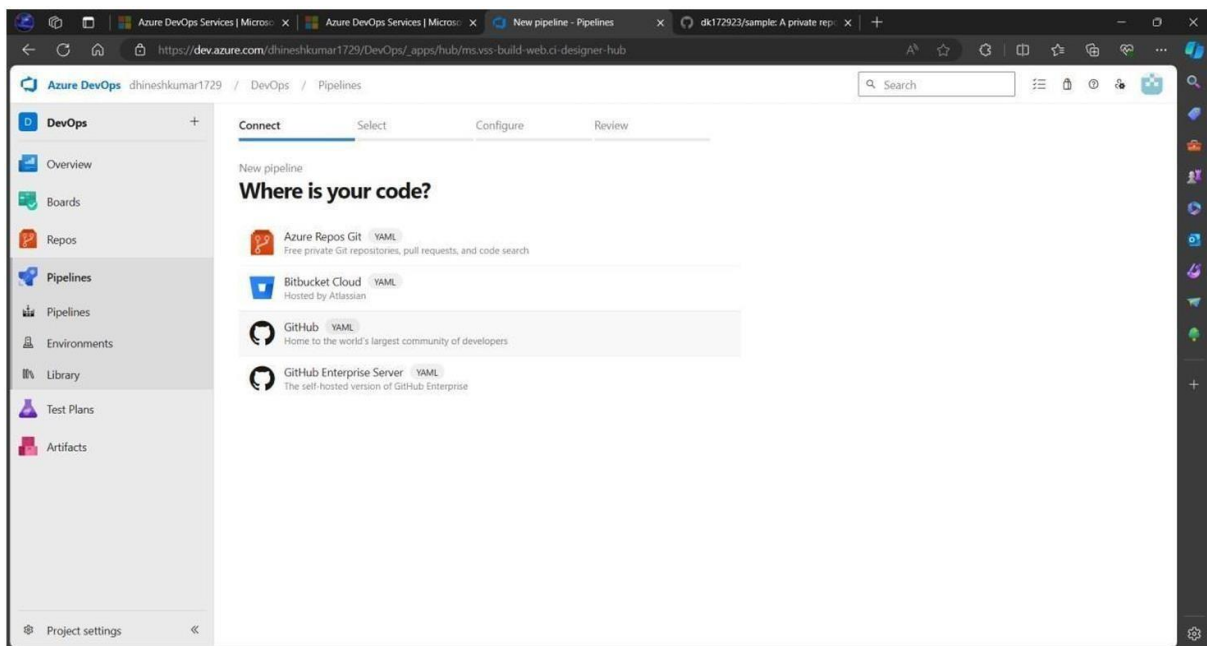


STEP 5: Create a pipeline for the created maven repository.

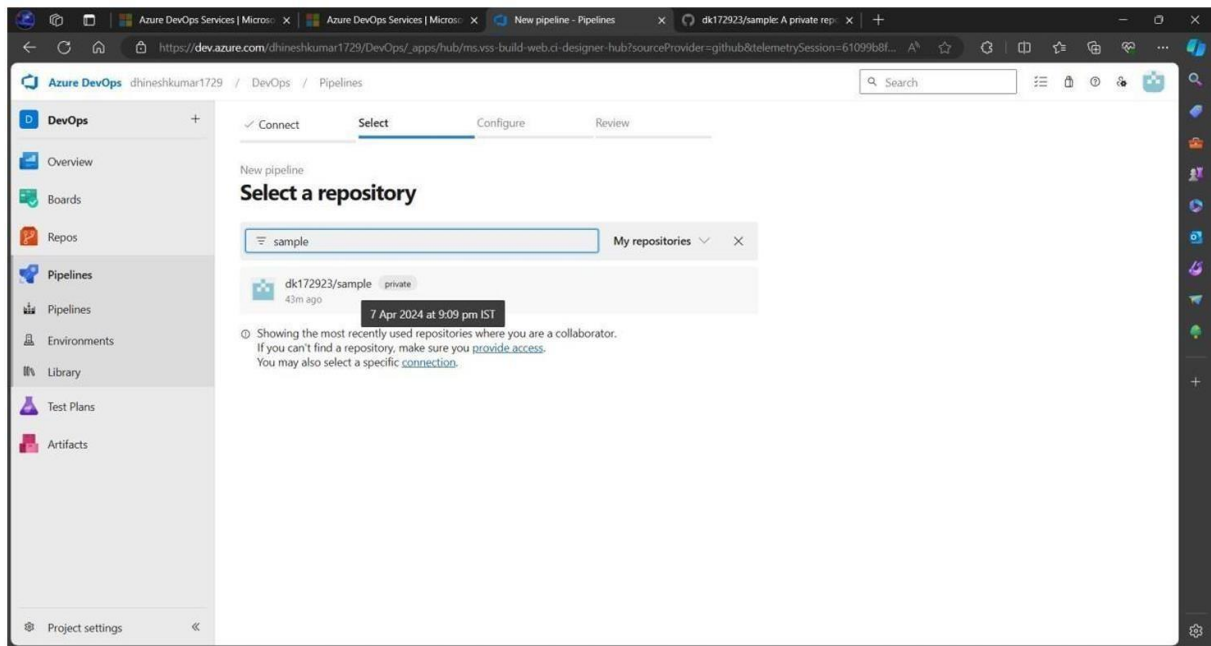
Click on Pipelines on the left pane of the Azure DevOps website.



1. Click on the Create Pipeline button.
2. Choose the GitHub YAML option.

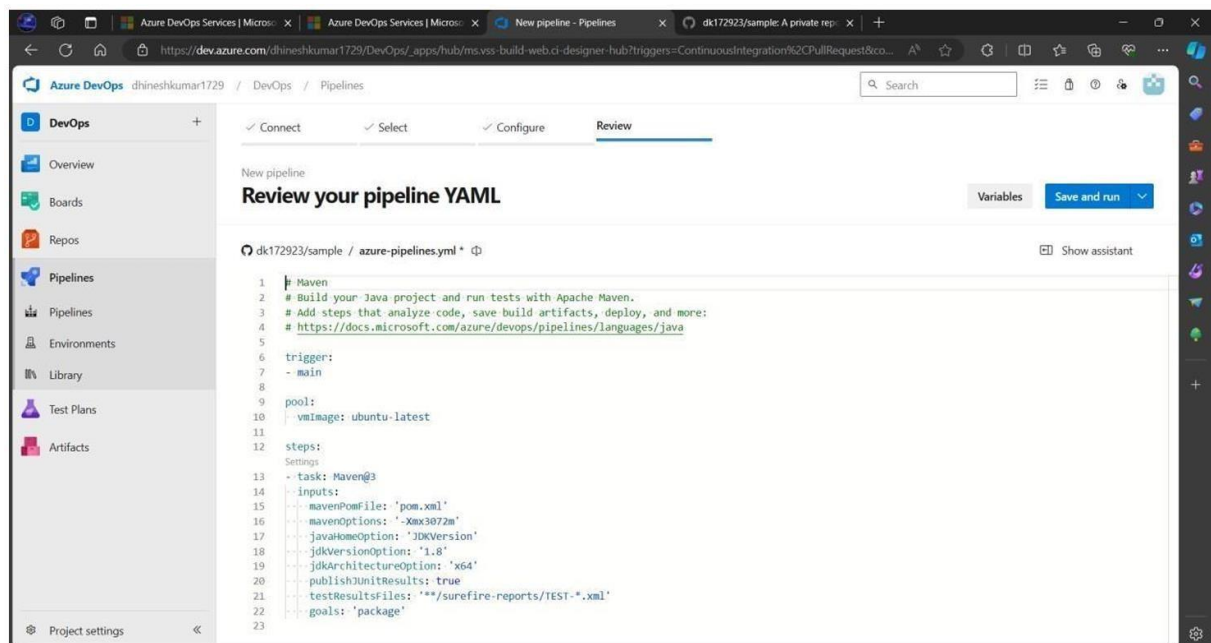


1. Select the created repository "sample" from the available repositories.
2. Give permissions if prompted to do so and sign in to your GitHub account if needed.



3. Choose Maven Pipeline from the given options.

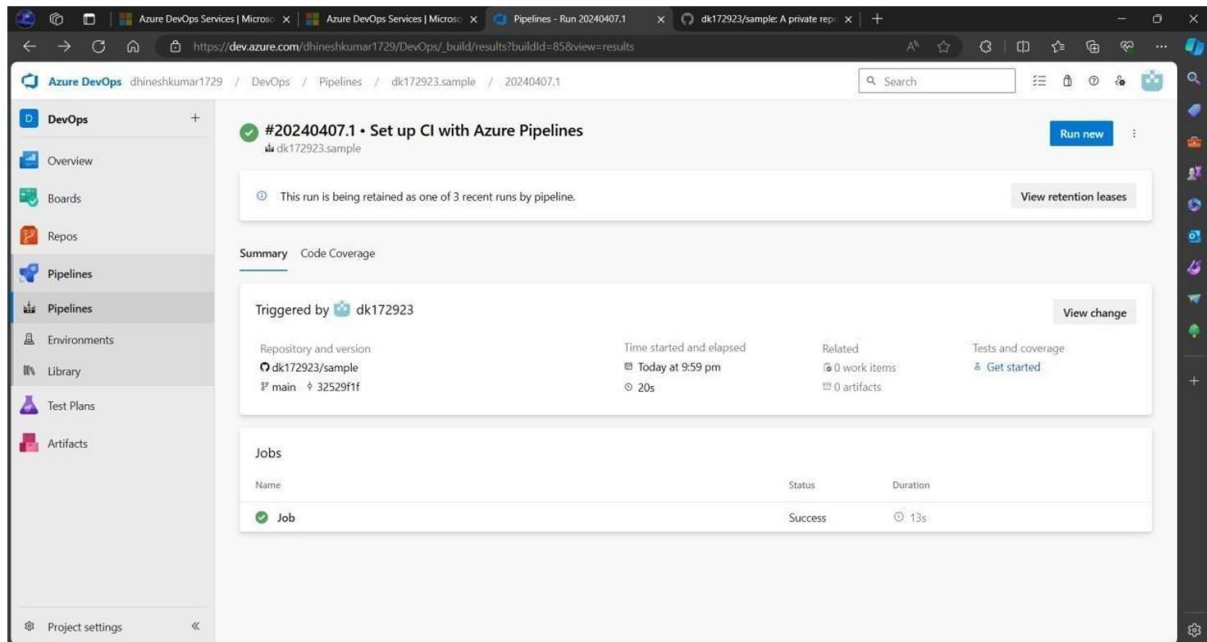
4. A YAML file is automatically created based on the configuration details found in the pom.xml file in the repository.



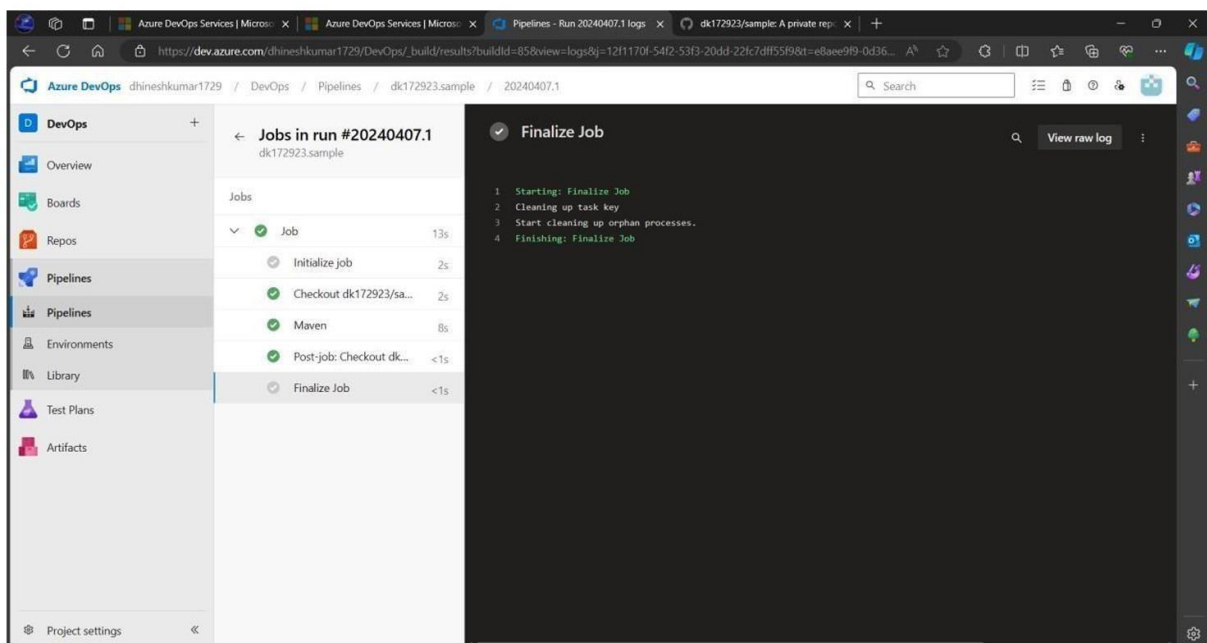
1. Press the Save and Run button.

2. Create a commit message and press the run button again.

3. Click on the created Job from the Jobs table.



A successful job completion would be like this.



Below is a detailed section that explains how to run unit tests and generate reports in an Azure DevOps pipeline after your Maven project has been built. This section explains what happens

during the Maven build process, how test results are generated, and how to publish these results as part of your pipeline execution.

Running Unit Tests and Generating Reports with Maven in Azure DevOps

When you build your Maven project using Azure Pipelines, the build process usually includes running unit tests with the Maven Surefire plugin. This plugin executes tests (typically written with JUnit) and produces test result files in XML format. Azure Pipelines can then pick up these XML files and present them as part of the build summary. Below are the steps and details to ensure that your unit tests are executed and the reports are published.

1. Maven Surefire Plugin and Test Reports

What Happens During the Maven Build?

- **Maven Surefire Plugin:**
When you run the command `mvn clean package` (or `mvn test`), the Surefire plugin automatically executes the unit tests found in the `src/test/java` directory.
- **Test Report Generation:**
By default, the Surefire plugin creates XML reports in the `target/surefire-reports/` directory. These files usually have names like `TEST-<TestClassName>.xml`.
- **Importance of Test Reports:**
These XML files contain detailed information on test execution, including the number of tests run, passed, failed, and any error messages or stack traces.

2. Configuring Your Azure Pipeline to Publish Test Results After your Maven build runs and tests are executed, you need to add a step in your Azure Pipeline YAML file that locates these test reports and publishes them in Azure DevOps. This is accomplished by using the **PublishTestResults@2** task.

YAML Configuration Example for Maven

Below is a sample snippet of a YAML pipeline configuration that includes both the Maven build step and a step to publish test results:

trigger:

- main

pool:

vmImage: 'ubuntu-latest'

steps:

Step 1: Maven Build and Test Execution

- task: Maven@3

inputs:

mavenPomFile: 'pom.xml' # Ensure your pom.xml is at the repository root

goals: 'clean package'

options: " # Add any additional Maven options if needed

Step 2: Publish Unit Test Results

- task: PublishTestResults@2

inputs:

testResultsFiles: '**/target/surefire-reports/TEST-*.xml'

mergeTestResults: true

testRunTitle: 'Maven Unit Test Results'

Explanation of the YAML Steps:

1. Maven Build Step:

- **Task:** Maven@3
- **Input:**
 - **mavenPomFile:** Points to your pom.xml.
 - **goals:** The goals clean package ensure that your code is cleaned, compiled, tested, and packaged. The Surefire plugin runs tests during this process.
- **Outcome:** Maven executes the tests and creates XML reports under the target/surefire-reports/ folder.

2. Publish Test Results Step:

- **Task:** PublishTestResults@2
- **Input:**
 - **testResultsFiles:** Uses a glob pattern (**/target/surefire-reports/TEST-*.xml) to find all XML test result files.
 - **mergeTestResults:** Set to true so that if there are multiple result files, they are merged into a single report.
 - **testRunTitle:** Provides a title for the test run that will appear in the Azure DevOps test results view.
- **Outcome:** Azure Pipelines reads the test result XML files and displays a summary in the pipeline's **Tests** tab. You'll be able to see metrics like the number of tests passed, failed, or skipped, and review detailed error messages for any failures.

3. Running and Verifying the Pipeline

After committing the YAML file to your repository, the pipeline is triggered (either automatically or manually):

1. Trigger the Pipeline:

- Once your YAML file is saved in your repository, Azure Pipelines will pick up the changes. If not automatically triggered, you can click **“Run pipeline”** manually.
- **Screenshot Tip:** Capture the pipeline run screen where you see the stages (e.g., Checkout, Maven Build, Publish Test Results).

2. Monitor the Build Output:

- Navigate to the “**Build**” or “**Logs**” section of the pipeline run.
- Confirm that the Maven task logs show the execution of tests and that the Surefire reports are created.

3. **Review Test Reports:**

- Once the build completes, click on the “**Tests**” tab (often found on the pipeline summary page) to review the detailed test results.
- **Screenshot Tip:** Capture the test report summary, showing the number of tests executed, passed, and failed. You may also capture detailed logs for any failed tests.

4. **Troubleshooting Tips**

- | | | | |
|-------------|-------------|----------------|---------------|
| • No | Test | Results | Found: |
|-------------|-------------|----------------|---------------|

If you see a warning that no test results were found, verify that:

- Your tests are indeed being executed (check the Maven build log).
- The path in testResultsFiles correctly matches the location and naming pattern of the generated XML files.

- Test Failures:**

If tests fail, review the published test report for detailed error messages and stack traces. Adjust your tests or code as necessary and re-run the pipeline.

Experiment 11: Creating Release Pipelines: Deploying Applications to Azure App Services, Managing Secrets and Configuration with Azure Key Vault, Hands-On: Continuous Deployment with Azure Pipelines

1. Overview

In this experiment, you will:

- **Deploy your build artifact** (e.g., a JAR or WAR file from a Maven/Gradle project) to an Azure App Service.
- **Manage secrets and configuration** securely using Azure Key Vault.
- **Set up a release pipeline** in Azure DevOps that automatically deploys your application when a new artifact is available (continuous deployment).

This experiment bridges the gap between build automation (CI) and release automation (CD) while ensuring secure management of sensitive information.

2. Prerequisites

Before you begin, ensure that you have:

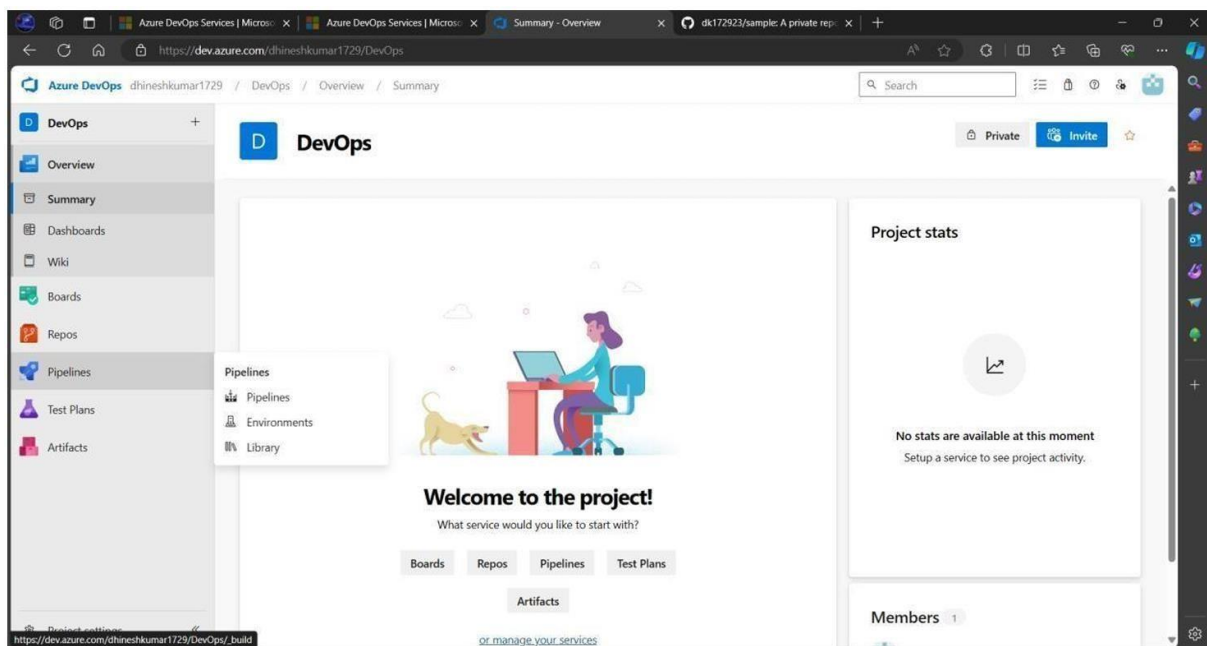
- An **Azure DevOps account** with a project set up (see Experiment 9 and Experiment 10).
- A **build artifact** (e.g., your Maven/Gradle artifact) available from a build pipeline.
- An **Azure Subscription** with an **Azure App Service** instance already created to host your application.
- An **Azure Key Vault** instance created in your Azure Subscription for storing secrets (e.g., connection strings, API keys).
- Appropriate permissions to create and manage resources in Azure DevOps and your Azure Subscription.

3. Creating a Release Pipeline in Azure DevOps

A. Create a New Release Pipeline

1. **Log in to Azure DevOps:**

- Open your web browser and navigate to your Azure DevOps project (e.g., <https://dev.azure.com/YourOrganization>).
2. **Navigate to the Releases Section:**
- In the left-hand menu, click on “**Pipelines**” and then “**Releases**”.
3. **Create a New Pipeline:**
- Click on “**New pipeline**”.
 - When prompted, select “**Empty job**” (or start with a template if one suits your needs).



B. Add an Artifact

1. **Link Your Build Artifact:**
- In the release pipeline editor, click on “**Add an artifact**”.
 - **Source Type:** Choose the source (e.g., “Build”).
 - **Source (Build Pipeline):** Select the build pipeline that produces your Maven/Gradle artifact.
 - **Default Version:** Use the latest version (or specify a branch/tag as needed).
 - Click “**Add**”.

C. Define a Stage for Deployment

1. **Add a New Stage:**

- Click on **“Add a stage”** and select **“Empty job”**.
- Rename the stage (e.g., Development or Production).

2. **Configure the Stage:**

- Click on the stage to open its settings.
- In the **“Tasks”** view for the stage, click **“+”** to add a new task.

4. **Deploying to Azure App Services**

A. **Add the Azure App Service Deploy Task**

1. **Add Deployment Task:**

- In the stage tasks, click on **“+ Add”** and search for **“Azure App Service Deploy”**.
- Click **“Add”** to include it in your stage.

2. **Configure the Azure App Service Deploy Task:**

- **Azure Subscription:**
 - Click on **“Authorize”** (if required) and select your Azure subscription.
- **App Service Type:** Choose **“Web App on Windows”** or **“Web App on Linux”** based on your deployment target.
- **App Service Name:** Enter the name of your Azure App Service instance.
- **Package or Folder:**
 - Specify the path to your build artifact (for example, \$(System.DefaultWorkingDirectory)/_HelloMaven-CI/drop/HelloMaven-1.0-SNAPSHOT.jar or the folder containing your package). Adjust the path based on your artifact’s location.
- **Deployment Options:** Configure any additional options (e.g., deployment slot, resource group) as needed.

5. **Managing Secrets and Configuration with Azure Key Vault**

A. **Create and Configure an Azure Key Vault (in Azure Portal)**

1. **Create a Key Vault (if not already created):**

- Log in to the [Azure Portal](#).
- Click on **“Create a resource”** and search for **“Key Vault”**.

- Follow the prompts to create a new Key Vault (enter a name, select subscription, resource group, and region).

2. Add Secrets to Your Key Vault:

- Once the Key Vault is created, navigate to it.
- Click on “**Secrets**” and then “**Generate/Import**”.
- Create new secrets (e.g., DBConnectionString, APIKey) and note their names.

B. Integrate Key Vault with Azure DevOps

1. Create a Variable Group Linked to Key Vault:

- In your Azure DevOps project, navigate to “**Pipelines**” and then “**Library**”.
- Click on “**+ Variable group**”.
- Name the variable group (e.g., KeyVault-Secrets).
- Enable “**Link secrets from an Azure Key Vault as variables**”.
- Click “**Authorize**” and select your Azure subscription.
- Select your Key Vault from the dropdown list.
- Choose the secrets you want to import into the variable group.
- Save the variable group.

2. Use the Variable Group in Your Release Pipeline:

- In your release pipeline, click on the “**Variables**” tab.
- Click “**Variable groups**” and then “**Link variable group**”.
- Select the variable group you created (KeyVault-Secrets).
- Now, you can reference these secrets as variables in your deployment tasks (for example, \$(DBConnectionString)).

6. Enabling Continuous Deployment

A. Configure Continuous Deployment Trigger

1. Enable Continuous Deployment:

- In your release pipeline, click on the “**Triggers**” tab.
- Under “**Artifact filters**” or “**Continuous deployment trigger**”, enable the toggle to allow the release pipeline to trigger automatically when a new build artifact is available.
- Optionally, you can specify branch filters or artifact version filters.

2. **Save the Release Pipeline:**

- Click “**Save**” to apply all changes.

B. Test the Continuous Deployment Pipeline

1. **Trigger a Build:**

- Commit a change to your code repository to trigger your build pipeline (or trigger it manually).
- Verify that the build pipeline creates a new artifact.

2. **Automatic Release:**

- Once the new artifact is published, the release pipeline should automatically trigger a new release.
- Monitor the release pipeline execution to ensure that:
 - The artifact is deployed to the Azure App Service.
 - The deployment task uses the configuration and secrets from Key Vault.

Experiment 12: Practical Exercise and Wrap-Up: Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A

1. Overview

In this experiment, you will create an end-to-end DevOps pipeline that demonstrates the following processes:

- **Version Control:** Code is maintained in a Git repository (e.g., GitHub or Azure Repos).
- **Continuous Integration (CI):**
 - A CI tool (Jenkins and/or Azure Pipelines) automatically checks out the code, builds it using Maven/Gradle, runs unit tests, and archives the build artifact.
- **Artifact Management:** The artifact (e.g., a JAR file) is archived and made available for deployment.
- **Continuous Deployment (CD):**
 - Deployment automation is handled either by an Ansible playbook or an Azure Release pipeline, deploying the artifact to a target environment (such as an Azure App Service or a local server).
- **Secrets and Configuration Management:** Securely manage configuration details and secrets using Azure Key Vault.
- **Pipeline as Code:** Use YAML (for Azure Pipelines) or a Jenkinsfile (for Jenkins) to define your build and release processes.

After setting up and running the pipeline, we will discuss best practices for designing and maintaining such pipelines and open the floor for a Q&A discussion.

2. Prerequisites

Before starting, ensure you have completed or have access to the following:

- **Source Code Repository:**
 - A Java project (e.g., “HelloMaven” or “HelloGradle”) hosted on GitHub or Azure Repos.

- The project should follow a standard structure (with pom.xml for Maven or build.gradle for Gradle, and appropriate src/main/java and src/test/java directories).
- **Jenkins and/or Azure DevOps Setup:**
 - Jenkins installed and configured on your local machine or a cloud server (refer to Experiments 5 and 6), or an Azure DevOps project set up with a build pipeline (Experiments 9 and 10).
- **Ansible Installed:**
 - Ansible is installed on your control machine (or Jenkins server) with a basic inventory file (see Experiment 7).
- **Azure Resources:** (Optional but recommended for cloud deployment)
 - An Azure App Service instance created to host your application.
 - An Azure Key Vault instance set up to store sensitive data (e.g., connection strings).
- **Access Credentials:**
 - Permissions to commit code to the repository.
 - Administrative access on Jenkins/Azure DevOps.
 - Proper permissions on Azure to deploy to App Services and to manage Key Vault secrets.

3. Step-by-Step Pipeline Setup

Step 1: Code Repository Preparation

1. **Ensure Your Project Is in Version Control:**
 - Navigate to your project folder (e.g., “HelloMaven”) and initialize Git if not already done:
 - `cd /path/to/HelloMaven`
 - `git init`
 - `git add .`
 - `git commit -m "Initial commit of HelloMaven project"`
 - Push the project to your remote repository (e.g., GitHub):
 - `git remote add origin https://github.com/yourusername/HelloMaven.git`
 - `git push -u origin main`

Step 2: Continuous Integration with Jenkins and/or Azure Pipelines

A. Jenkins CI Pipeline Setup

1. Create a New Jenkins Job:

- Log in to Jenkins and click “**New Item**”.
- Enter a name (e.g., HelloMaven-CI) and select “**Freestyle project**” or “**Pipeline**”.
- *Screenshot Tip:* Capture the new job creation screen.

2. Configure Source Code Management:

- Under the “**Source Code Management**” section, select “**Git**”.
- Enter your repository URL (e.g., `https://github.com/yourusername/HelloMaven.git`) and set branch specifier to `*/main`.

3. Add Build Steps:

- **For Maven Projects:**
 - Add a build step “Invoke top-level Maven targets” and set the goals to:
 - `clean package`
- **For Pipeline-as-Code (Jenkinsfile):**
 - Create a Jenkinsfile in your repository with stages for checkout, build, test, and archive. For example:

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git url: 'https://github.com/yourusername/HelloMaven.git',
            branch: 'main'
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }
  }
}
```

```

▪    }
▪    stage('Test') {
▪        steps {
▪            sh 'mvn test'
▪        }
▪    }
▪    stage('Archive') {
▪        steps {
▪            archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
▪        }
▪    }
▪ }
▪ }
```

4. Run the Jenkins Job:

- Click **“Build Now”** and monitor the console output. Ensure that the build is successful and that test reports are generated.

B. Azure DevOps Build Pipeline Setup

1. Create a New Pipeline:

- Log in to your Azure DevOps project.
- Navigate to “**Pipelines**” and click “**New pipeline**”.
- Select your repository source (GitHub or Azure Repos) and choose your repository.

2. Define Your YAML Pipeline:

- Use the following sample YAML for a Maven project:
- trigger:
- - main
-
- pool:
- vmImage: 'ubuntu-latest'
-
- steps:
- - task: Maven@3

- inputs:
- mavenPomFile: 'pom.xml'
- goals: 'clean package'
- - task: PublishTestResults@2
- inputs:
- testResultsFiles: '**/target/surefire-reports/TEST-*.xml'
- mergeTestResults: true
- testRunTitle: 'Maven Unit Test Results'

3. Run the Pipeline and Verify Test Reports:

- Commit and run the pipeline.
- Navigate to the “**Tests**” tab to view the summary of executed tests

Step 3: Artifact Management

1. Artifact Archiving in Jenkins:

- Ensure your Jenkins job archives the artifact (JAR file) using the “**Archive the artifacts**” post-build action.

2. Artifact in Azure Pipelines:

- The build task produces an artifact that can be downloaded or referenced by subsequent release pipelines.

Step 4: Deployment Automation with Ansible and Azure Release Pipeline

A. Using Ansible for Deployment

1. Write an Ansible Playbook:

- Create a file named deploy.yml:
- ---
- - name: Deploy Maven Artifact
- hosts: deployment
- become: yes
- tasks:
- - name: Copy the artifact to the target directory
- copy:

- `src: "/var/lib/jenkins/workspace/HelloMaven-CI/target/HelloMaven-1.0-SNAPSHOT.jar"`
- `dest: "/opt/deployment/HelloMaven.jar"`
- Adjust paths according to your environment.

2. **Configure Your Ansible Inventory:**

- Create or update your `hosts.ini` file:
- `[deployment]`
- `target-server ansible_host=your.server.ip ansible_user=yourusername`
- For a local deployment, you can use:
- `[deployment]`
- `localhost ansible_connection=local`

3. **Integrate Ansible into Your Jenkins/Azure Pipeline:**

- Add a post-build (or post-release) step to execute the Ansible playbook:
- `ansible-playbook -i /path/to/hosts.ini /path/to/deploy.yml`

B. Using Azure Release Pipeline to Deploy to Azure App Services

1. **Create a New Release Pipeline in Azure DevOps:**

- Navigate to “**Pipelines > Releases**”.
- Click “**New pipeline**” and select an empty job.

2. **Link Your Build Artifact:**

- Click “**Add an artifact**” and select your build pipeline as the source.

3. **Add a Deployment Stage for Azure App Service:**

- Create a new stage (e.g., Production).
- Add the “**Azure App Service Deploy**” task.
- Configure the task with your Azure subscription, target App Service, and the path to the artifact.

4. **Integrate Key Vault (Optional for Managing Secrets):**

- Create and link a Variable Group that pulls secrets from Azure Key Vault (refer to previous experiments).
- Reference these secrets in your deployment tasks (e.g., connection strings).

5. **Enable Continuous Deployment:**

- In the release pipeline’s triggers, enable continuous deployment so that a new release is automatically created when a new artifact is available.

6. Run the Release Pipeline and Verify Deployment:

- Trigger the release pipeline (either manually or automatically).
- Verify that the application is deployed to your target environment (e.g., by browsing to the Azure App Service URL or checking the deployment directory on the target server).

Step 5: End-to-End Pipeline Demonstration

1. Trigger a Complete Run:

- Make a change in your code repository (e.g., modify a welcome message in your Java application) and commit it.
- This should trigger the CI pipeline (Jenkins or Azure Pipelines), which builds, tests, and archives the artifact.
- The artifact triggers the CD process (via Ansible or Azure Release), and the application is deployed automatically.

2. Verify the Entire Workflow:

- Check the CI pipeline output (build success and test results).
- Confirm that the artifact is archived.
- Review the CD pipeline logs (deployment success, any post-deployment notifications, etc.).
- Optionally, log into the target environment and verify that the new version of your application is running.

Step 6: Discussion on Best Practices and Q&A

Best Practices:

- **Pipeline as Code:**
Use YAML (or a Jenkinsfile) to define your build and release pipelines. This allows you to version control your pipeline configuration alongside your code.
- **Automate Everything:**
Automate code checkout, builds, tests, artifact archiving, and deployments. Reduce manual interventions to minimize human error.

- **Idempotence:**

Ensure that your deployment scripts (whether Ansible playbooks or release tasks) are idempotent—running them multiple times produces the same result.

- **Secure Secrets Management:**

Use Azure Key Vault (or a similar tool) to securely store sensitive data (e.g., API keys, connection strings) and reference these values in your pipelines.

- **Monitoring and Logging:**

Integrate logging and monitoring into your pipeline. Review test reports, deployment logs, and set up notifications for build failures.

- **Modular and Scalable Design:**

Break down your pipeline into clear stages (checkout, build, test, deploy) and design it to handle multi-environment deployments (development, staging, production).

- **Continuous Improvement:**

Regularly review and refine your pipeline. Use metrics and feedback to optimize build times, reduce failures, and ensure high-quality releases.

Q&A Discussion Points:

- **Troubleshooting Pipeline Failures:**

What are common reasons for build/test failures, and how can you diagnose issues from the logs?

- **Handling Rollbacks:**

What strategies can be implemented for rolling back deployments if a release fails in production?

- **Scaling Pipelines:**

How do you handle scaling the pipeline for larger projects or multiple microservices?

- **Integrating Additional Tools:**

What additional tools (e.g., code quality analyzers, security scanners) could be integrated into this pipeline?

- **Real-World Challenges:**

Discussion of challenges faced in actual DevOps implementations and strategies to overcome them.

Open the Floor for Questions:

Encourage participants to ask questions regarding any part of the pipeline—from technical configuration details to high-level best practices.